

Algebraic and Computational Limits of LLM Guardrails

Joseph Robert Lopez

Abstract

LLM guardrails face four structurally distinct barriers: algebraic blindness arising from syntactic monoid aperiodicity (unconditional); an illustrative information-theoretic lower bound (Fano-type, under a uniformity assumption); NP-hardness of instantiation verification; and structural transfer via free-category functoriality (unconditional) combined with string-level indistinguishability under a semantic-opacity assumption on symbol naming. Together these results characterize why inference-layer defenses are necessary but insufficient.

We operationalize these barriers through five attack vectors. V1–V4 (*homomorphic reasoning*: decomposition, zero-knowledge pipelines, Tree-of-Thought solving over abstract grammars, and encoding bootstrap) exploit the information-theoretic and computational barriers against abstraction-based attacks. V5 (*modular counting bypass*) exploits algebraic blindness: we prove that all substring-matching regex guardrails have aperiodic syntactic monoids and are therefore provably blind to any payload encoded using modular counting. Empirically, V3 yields a mean yield of 0.466 for BFS, 0.172 for random-beam, and 0.122 for LLM-guided Tree-of-Thought ($N=50$, seeds 0–49, $p<0.001$); BFS dominates, as exhaustive search over small synthetic grammars outperforms LLM heuristic pruning. We extracted syntactic monoids from a corpus of 142 patterns drawn from twelve sources — 100 patterns shipped by nine third-party open-source guardrail projects and 42 patterns assembled from three author-curated pattern sets; 100% are aperiodic, and the MOD₂ bypass construction succeeds against all aperiodic patterns. A 376-line proof-of-concept with three execution mediums validates all five vectors. We conclude that inference-layer guardrails are necessary but insufficient, and that effective defense must migrate to the execution layer where concrete artifacts become observable.

1 Prelude: The Guardrail Problem

This section assumes no prior knowledge of formal language theory, circuit complexity, or abstract algebra; formal definitions follow in §5.

Every major AI deployment protects itself at the first layer with a regex gate: a list of patterns scanned against the user’s message before the model sees it. For the threats

these gates were designed to catch—known keywords, obvious phrasings, copy-pasted attack strings—they work. The question this paper asks is: what are they *structurally* unable to see?

Each pattern compiles to a finite-state machine (DFA) with a bounded state space and no external memory. Such a machine carries a mathematical fingerprint—its *syntactic monoid* (§5.1)—that captures every distinction the machine can make; two inputs the monoid identifies are, from the gate’s perspective, the same string, and this identification is a structural consequence of the fixed state space rather than a bug. For substring-matching guardrails the syntactic monoid is *aperiodic*: the machine can match but cannot count.

Three classical theorems chain this observation into a hard ceiling. Schützenberger (1965) shows that aperiodic monoids recognize exactly the *star-free* languages [30]. Barrington, Compton, Straubing, and Thérien (1992) show that star-free languages correspond exactly to AC⁰ circuits [3]. Furst, Saxe, and Sipser (1981) and Håstad (1987) show that AC⁰ cannot compute parity [9, 14]. Composing the chain: keyword gates provably cannot decide whether a count is even or odd, regardless of how they are tuned. An attacker who hides a payload across alternating positions—the filler on even indices, the payload on odd—slips past the gate unharmed; the decoded payload is algebraically invisible because decoding it is a parity operation the gate cannot perform. The conclusion is not a conjecture.

This paper characterizes this blind spot precisely (§6), measures it across 142 production patterns (§8.1), identifies four additional attack vectors that exploit related structural barriers (§7), and constructs a defense that breaks the ceiling (§10.3).

2 Introduction

Production LLM deployments stack safety measures in depth: a regex pre-filter blocks known-harmful patterns before the request reaches the model, a neural classifier operating at inference time catches a broader distributional range of harmful intent, and an inference-time monitor inspects generated outputs before they are returned to the user. Each layer is individually sound for the threats it was designed to address. We show that each layer has a structural limitation provable from first principles, and that together these limitations leave a systematic gap an

adversary can exploit without model access, fine-tuning, or gradient computation.

The core observation. We observe that an LLM solving abstract formal systems produces correct real-world procedures after offline instantiation, without the guardrail ever observing harmful content. The gap between symbolic manipulation and semantic grounding—often cited as a limitation of language models—becomes precisely the mechanism by which content-based guardrails are bypassed: the content visible to the guardrail is indistinguishable from legitimate formal reasoning.

2.1 Overview of Results and Paper Organization

We use *homomorphic* in its algebraic sense throughout: the interpretation functor $\hat{I}$ (Proposition 6.17) is a structure-preserving map from the free category of abstract derivations to the category of domain operations—a homomorphism of categories. The attack exploits this homomorphism: reasoning performed in the abstract category maps faithfully to operational sequences in the target domain.

Our contributions, organized by section, are:

1. **Algebraic impossibility** (sections 5 and 6): substring-matching regex guardrails have aperiodic syntactic monoids (Theorem 6.1) and are therefore provably blind to all modular counting encodings (Theorem 6.4; Corollary 6.5).
2. **Information-theoretic lower bound** (section 6.5): abstraction functions destroy safety-critical information; under a uniform prior over concrete artifacts, Fano’s inequality (Proposition 6.12) yields an illustrative error floor for any lifted guardrail. We present this as a motivating lower bound under simplifying assumptions, not as an unconditional impossibility; the algebraic results below are the core contribution.
3. **Computational impossibility** (section 6.6): verifying whether an abstract program has a dangerous instantiation is NP-complete (Proposition 6.14).
4. **Structural transfer and string-level indistinguishability** (sections 6.7 and 6.8): abstract derivations transfer faithfully to real domains via free-category functoriality (Proposition 6.17, unconditional); and under a semantic-opacity assumption on symbol naming, the homomorphic-reasoning attack’s prompts are string-level indistinguishable from legitimate formal-reasoning exercises (Observation 6.20). We present the indistinguishability result as a structural observation rather than an independent impossibility: it closes the argument *once* opacity holds and an attack of this form exists (empirically demonstrated in §7.3–7.4 and visualized in Table 8).
5. **Five attack vectors validated empirically** (section 7): V1–V4 demonstrate homomorphic reasoning; V5

demonstrates modular counting bypass.

6. **142-pattern regex corpus analysis plus validation** (section 8.1): confirms 100% aperiodicity across 142 patterns drawn from twelve sources — 100 patterns shipped by nine third-party open-source guardrail projects and 42 patterns from three author-assembled sets, matching the breakdown in CACM v14 Appendix B.1 — with verified MOD₂ bypass constructions.
7. **Defense recommendations grounded in formal results** (section 10): each impossibility theorem maps to a specific layer of a defense-in-depth stack; as a concrete instantiation of the Class C defense architecture, we release `secrets-router` (§11.4, MIT-licensed, ~800 lines), an MCP server that inverts the interpretation-table asymmetry exploited by homomorphic-reasoning attacks.

Nature of the contribution. We use classical tools—syntactic monoids, star-freeness, Fano’s inequality, free categories, Krohn–Rhodes decomposition—applied to the guardrail security setting. The mathematical novelty lies in the *application* and in the combination of four independent barriers (algebraic, information-theoretic, computational, and structural) into a single unified impossibility argument, not in the underlying tools themselves. We state this up front to situate the contribution accurately: this is a security paper that uses formal language theory, not a formal language theory paper.

Scope. The algebraic blindness result (§5) applies specifically to regex guardrails and is unconditional. Transformer-based (neural) guardrails operate in TC⁰ [5, 25], which *can* compute MOD_p, so the modular counting blind spot does not transfer. However, neural guardrails face the information-theoretic and computational barriers (§§6.5–6.6) and the structural indistinguishability result (§6.8), which apply at any layer that inspects only LLM-visible content.

3 Background and Related Work

3.1 Jailbreaking Taxonomies

Wei et al. [36] identify *mismatched generalization* as a failure mode in which safety training does not cover the full distribution of capability-eliciting inputs. Our abstraction bypass is a systematic instance: the model’s formal reasoning capabilities generalize beyond the distribution on which safety training was performed. Zou et al. [42] demonstrate gradient-based adversarial suffixes; our attacks operate in semantic space rather than weight space and require no model access whatsoever. Xu et al. [11]

provide an empirical taxonomy of guardrail bypass strategies spanning encoding, role-play, and indirect instruction attacks; our algebraic results explain why the encoding-based attacks in that taxonomy are provably undetectable by regex guardrails. The SoK by Li et al. [21] systematizes evaluation of jailbreak guardrails, highlighting the persistent gap between guardrail effectiveness and attacker capability—a gap our impossibility results formalize.

Esmradi et al. [8] introduce h4rm3l, a composable jailbreak language in which individual text-level transformations are composed to produce effective attacks. Our homomorphic reasoning framework generalizes composable string transformations: rather than composing text-level encodings, we compose structure-level abstractions whose compositional behavior is guaranteed by functoriality (Proposition 6.17).

Huang et al. [12] demonstrate that large reasoning models achieve a 97% attack success rate when deployed as autonomous jailbreak agents, confirming that LLM reasoning capability itself is the primary attack surface. This directly parallels our core thesis: the attack does not exploit model miscalibration but the model’s genuine reasoning competence applied to abstractions the guardrail cannot interpret.

3.2 Encoding and Structural Attacks

StructuralSleight [20] achieves 94.62% attack success on GPT-4o via uncommon text-encoded structures (ASCII art, cipher text, etc.), demonstrating that encoding diversity defeats content filters. Our algebraic results explain *why*: any encoding that requires modular counting to decode lies outside the AC^0 closure of any aperiodic guardrail language (Theorem 6.4).

Boucher et al. [4] demonstrate imperceptible attacks via Unicode manipulation—inserting invisible characters between payload characters so that the guardrail sees an interrupted substring. Zero-width character insertion is an instance of MOD_2 interleaving with a null filler character; our Corollary 6.5 proves it cannot be patched within the regex paradigm.

The Content Concretization attack [35] traverses the abstraction hierarchy in the opposite direction: given an abstract harmful specification, the attacker prompts the LLM to supply concrete details. We proceed concrete-to-abstract—encoding a concrete harmful task as an abstract formal system—and the interpretation happens offline. The two approaches are complementary instances of the same general principle that abstraction boundaries defeat content inspection.

DictAttack [37] embeds malicious intent in grammar-level schema rules while maintaining benign surface prompts, achieving 94–99% attack success rates across multiple LLMs. This structured-output attack exploits the

same abstraction boundary our V3 and V4 vectors formalize: the gap between surface-level content filtering and structural information encoding. Our contribution is the formal impossibility proof that regex guardrails *cannot* close this gap, whereas DictAttack provides empirical evidence of its exploitability.

S2C [34] is independent concurrent work (March 2026) that manipulates semantic reconstruction during inference across multiple semantic dimensions, arriving at a complementary abstraction-based framing through a different research path. Where S2C operates within a single LLM inference pass, our pipeline distributes the harmful reasoning across multiple agents and an offline interpretation table; both works independently identify abstraction boundaries as the core attack surface.

3.3 Computational and Algebraic Impossibility

Ball et al. [2] prove that under standard cryptographic assumptions, adversarial prompts are computationally indistinguishable from benign ones for any efficient filter; our results are complementary—algebraic (assumption-free) blindness against modular counting for regex guardrails (Theorem 6.4), and under an opacity assumption on symbol naming, string-level indistinguishability for the homomorphic-reasoning construction (Observation 6.20). Kumar et al. [18] give certified defenses via randomized smoothing against ℓ_p -bounded perturbations; our attack is a structure-preserving map that lies outside the smoothing distribution’s support. Dong et al. [19] establish a thoroughness/cost no-free-lunch tradeoff; our results show that unbounded cost cannot patch the regex blindness of Theorem 6.4. R2-Guard [39] replaces regex with chain-of-thought reasoning, operating in TC^0 and escaping aperiodic blindness but remaining subject to the information-theoretic and structural indistinguishability barriers of §§6.5–6.8.

The homomorphic-reasoning channel is a *steganographic channel* [15]: harmful intent is hidden in a legitimate-formal-reasoning cocontext. In the Hopper–Langford–von Ahn formalism, perfect steganographic security requires that no polynomial-time distinguisher exceed negligible advantage; Observation 6.20 establishes *perfect* security under Assumption 6.19 at the string-set level (zero advantage, not merely negligible), rendering the distinguisher’s task information-theoretically impossible rather than computationally hard.

3.4 Formal Languages and Neural Networks

The transformer complexity-theoretic literature places log-precision transformers in TC^0 [5, 24, 25], which *can* compute modular predicates—hence neural guardrails are not subject to the aperiodic blindness of the regex layer but remain subject to the other barriers (§10.2). Hahn [13]

provides circuit-theoretic evidence for a related qualitative boundary on self-attention over periodic languages. Our empirical syntactic monoid computation follows the same minimal-DFA extraction pipeline used by Weiss et al. [38] to extract DFAs from RNNs via Angluin’s L^* [1].

3.5 Indirect Prompt Injection

Greshake et al. [10] establish that LLM-integrated applications blur the boundary between data and instructions, enabling an attacker to inject control flow via data channels. Our V2 (zero-knowledge pipeline) is a specific instance of this pattern: the “data” is a base64-encoded configuration that contains harmful parameters the agent processing it never reads. The theoretical underpinning differs—Greshake et al. exploit instruction/data confusion, while V2 exploits the information destruction formalized in Proposition 6.12—but the practical attack pattern is closely related.

3.6 Tool-Chain Composition Attacks

Zhou et al. [40] formalize the STAC (Simple Tool-Augmented Chain) attack class, in which individually benign tool calls compose into harmful pipelines—achieving $> 90\%$ attack success rate across 483 attack cases. Our V1 (decomposition) and V2 (zero-knowledge pipeline) are specific instances of this general pattern. The distinction is that STAC provides a large-scale empirical evaluation of tool-chain composition, while our contribution is the formal proof (Proposition 6.12, Proposition 6.14) that no inference-layer guardrail can detect such compositions without inspecting runtime state.

4 Threat Model and Formalization

4.1 Attacker and Defender Model

Attacker capabilities. The attacker has access to: (a) an LLM-based coding assistant with bash execution (e.g., Claude Code, Copilot, Cursor); (b) local LLM infrastructure (e.g., Ollama with open-weight models) for offline grammar solving without guardrail exposure; (c) standard programming tools (Python, AST manipulation, base64 encoding).

Defender capabilities. The guardrail system operates at LLM inference time via RLHF training, system prompt rules, and output filters. It inspects the content visible to the LLM—prompts, generated code, tool calls—but not runtime arguments, environment variables, base64-encoded data in files the agent does not read, or offline interpretation tables.

Attacker goal. Cause the LLM to produce computation that, when instantiated with attacker-controlled parameters or interpreted via an attacker-held interpretation table, performs a task the guardrail would refuse if asked directly. The LLM must never observe the harmful intent in plaintext during any inference call subject to guardrail inspection.

Out of scope. Attacks requiring model weight access, fine-tuning, or gradient computation. Attacks against execution-layer defenses (sandboxing, network ACLs). Social engineering of human reviewers.

4.2 Abstraction-Based Formalization

Definition 4.1 (Abstraction function). Let R be a finite set of concrete artifacts and M a finite set of abstract types. An *abstraction function* $U: R \rightarrow M$ maps each concrete artifact to its abstract type (e.g., $U(\text{streaming.example}) = \text{“video endpoint”}$, $U(\text{C}_9\text{H}_8\text{O}_4) = \text{“acetyl compound”}$).

We do not require U to be surjective; we only use the existence of the fiber structure defined below.

Definition 4.2 (Fiber). For $m \in M$, the *fiber* $U^{-1}(m) = \{r \in R : U(r) = m\}$ is the set of all concrete artifacts mapping to abstract type m . A fiber is *mixed* if it contains both allowed ($G(r) = 0$) and denied ($G(r) = 1$) artifacts under the ground-truth guardrail $G: R \rightarrow \{0, 1\}$.

Definition 4.3 (Lifted guardrail). A *lifted guardrail* is a function $G': M \rightarrow \{0, 1\}$ that approximates the ground-truth guardrail G through the abstraction layer: the system classifies concrete artifact r by evaluating $G'(U(r))$ rather than $G(r)$ directly.

The existence of mixed fibers is the root cause of the information-theoretic lower bound in Proposition 6.12: if a fiber $U^{-1}(m)$ contains both safe and unsafe concrete artifacts, no lifted guardrail G' can correctly classify all of them without access to r itself—but the guardrail operating at the abstraction layer sees only $m = U(r)$.

4.3 Guardrails as Language Recognizers

Let Σ be the finite alphabet of LLM prompt characters.

Definition 4.4 (Regex guardrail). A *regex guardrail* is a function $G: \Sigma^* \rightarrow \{\text{allow}, \text{block}\}$ defined by a finite set of regular expression patterns $\{r_1, \dots, r_k\}$ applied via substring search:

$$G(w) = \begin{cases} \text{block} & \text{if } \exists i, u, v, x : \\ & w = uxv, x \in L(r_i), \\ \text{allow} & \text{otherwise.} \end{cases}$$

Let $L_{\text{safe}} = \{w \in \Sigma^* : G(w) = \text{allow}\}$ be the set of strings the guardrail permits.

Definition 4.5 (Blocked language). The *blocked language* of G is the complement $L_G = \Sigma^* \setminus L_{\text{safe}} = \Sigma^* \cdot (\bigcup_{i=1}^k L(r_i)) \cdot \Sigma^*$. This is a regular language (closed under concatenation with Σ^* and finite union).

Definition 4.6 (Guardrail blindness to encoding class). Let $\mathcal{E}$ be a class of invertible encodings $\phi: \Sigma^* \rightarrow \Sigma^*$. A guardrail G is *blind to encoding class* $\mathcal{E}$ if for every $\phi \in \mathcal{E}$:

1. $\phi(L_{\text{blocked}}) \cap L_G = \emptyset$ (encoded harmful strings pass the guardrail), and
2. there is no regular language L' with aperiodic syntactic monoid such that $\phi(L_{\text{blocked}}) \subseteq L' \subseteq \overline{L_{\text{safe}}}$ (no aperiodic regex can separate encoded-harmful from genuinely-benign strings).

Condition (1) states that the encoding evades the current guardrail; condition (2) states that the evasion cannot be patched by adding any regex whose syntactic monoid is aperiodic—which by §5 is the entire aperiodic regex class. This pair of conditions connects the regex language-recognizer formalization to the abstraction-based formalization of §4.2: regex guardrails are a specific instance of lifted guardrails operating on string content, and their fibers are the equivalence classes of the syntactic congruence $\equiv_{L_G}$.

5 Algebraic Foundations

This section recalls the classical algebraic and circuit-theoretic machinery underlying the impossibility results of §6. All results here are classical; full developments appear in Pin [28], Eilenberg [7], and Straubing [32].

5.1 Syntactic Monoids

For a finite alphabet Σ and regular language $L \subseteq \Sigma^*$, the *syntactic congruence* $\equiv_L$ identifies two strings when they behave identically in every left-right context: $u \equiv_L v$ iff $xuy \in L \Leftrightarrow xvy \in L$ for all $x, y \in \Sigma^*$.

Definition 5.1 (Syntactic monoid). The *syntactic monoid* of L is $M(L) = \Sigma^* / \equiv_L$, with the operation inherited from string concatenation. It is finite [26, 27], isomorphic to the transition monoid of the minimal DFA for L , and the smallest monoid recognizing L up to division [7, 28]—hence computable from the minimal DFA by standard monoid enumeration.

5.2 Aperiodicity and Star-Freeness

Definition 5.2 (Aperiodic monoid). A monoid M is *aperiodic* (equivalently, *group-free*) if it contains no nontrivial subgroup: for all $m \in M$, there exists $n \geq 1$ with

$m^n = m^{n+1}$. Cyclic groups $\mathbb{Z}/p\mathbb{Z}$ are the prototypical non-aperiodic monoids; aperiodic monoids cannot replicate their cyclic behavior.

Theorem 5.3 (Schützenberger 1965, McNaughton–Papert 1971 [23, 30]). *For a regular language L , the following are equivalent: (i) $M(L)$ is aperiodic; (ii) L is star-free—definable from Σ , $\emptyset$, $\{\varepsilon\}$, and single letters using Boolean operations and concatenation, without Kleene star; (iii) L is definable in first-order logic with linear order, $\text{FO}[<]$.*

Star-freeness and $\text{FO}[<]$ -definability are “counting-free”: they express positional and ordering constraints but not modular periodicity.

5.3 Circuit Complexity and Modular Counting

AC^0 is the class of languages recognized by families of constant-depth, polynomial-size Boolean circuits with unbounded fan-in AND/OR gates—the computations that parallelize without counting.

Theorem 5.4 (Barrington–Compton–Straubing–Thérien 1992 [3]). *A regular language is in AC^0 iff its syntactic monoid is aperiodic.*

Theorem 5.5 (Furst–Saxe–Sipser 1981, Håstad 1987 [9, 14]). $\text{PARITY} \notin \text{AC}^0$; more generally, $\text{MOD}_p \notin \text{AC}^0$ for any prime p , where $\text{MOD}_p = \{w \in \{0, 1\}^* : |w|_1 \equiv 0 \pmod{p}\}$.

Chaining Theorems 5.3–5.5: aperiodic $M(L_G) \Rightarrow L_G \in \text{AC}^0 \Rightarrow L_G$ cannot compute MOD_p , making the guardrail blind to any payload encoding whose decoder requires modular counting.

5.4 Krohn–Rhodes Decomposition

Theorem 5.6 (Krohn–Rhodes 1965 [17]). *Every finite monoid divides a wreath product of simple groups and aperiodic semigroups; the simple groups appearing in the decomposition (the group components) are uniquely determined up to isomorphism.*

The group components of $M(L_G)$ form a *complete spectrum of algebraic blind spots*: by the variety-theoretic characterization of Eilenberg [7] and Straubing [32], L_G can potentially express MOD_p only if some group component has order divisible by p ; the guardrail is provably blind to all other primes. When $M(L_G)$ is aperiodic (no group components) blindness is total—the case for every substring-matching regex guardrail, as established by Theorem 6.1.

6 Impossibility Results

6.1 Substring-Matching Patterns Are Aperiodic

Theorem 6.1 (Substring aperiodicity). *Let r be a regular expression built from literals, character classes, bounded repetition, alternation, optional groups, dot, and Kleene star or plus applied to character classes. Let $L_r = \Sigma^* \cdot L(r) \cdot \Sigma^*$ be the language of strings containing a substring matching r . Then the syntactic monoid $M(L_r)$ is aperiodic.*

Proof. We show L_r is star-free, hence aperiodic by the equivalence in Theorem 5.3.

Star-free closure under substring wrapping. For any regular language K over Σ , the language $\Sigma^* \cdot K \cdot \Sigma^*$ is star-free whenever K is star-free. This follows directly from the closure of star-free languages under concatenation [28]: $\Sigma^* = \emptyset$ is star-free, and the concatenation of three star-free languages is star-free. It therefore suffices to show that $L(r)$ itself is star-free for each building-block construction.

Literals and character classes. A single character $a \in \Sigma$ defines $\{a\}$, trivially star-free. A character class $[c_1 c_2 \dots c_k]$ is $\{c_1\} \cup \dots \cup \{c_k\}$, a finite union of singletons, hence star-free. The PCRE shorthands $\backslash s$, $\backslash d$, $\backslash w$ and their negations $\backslash S$, $\backslash D$, $\backslash W$ are character classes (they expand to finite unions of literal characters in any concrete alphabet), so Kleene $^*/+$ applied to them remains within this clause.

Bounded repetition. $r^{\{n\}}$ (exactly n repetitions) is the n -fold concatenation of $L(r)$, star-free by induction on n using closure under concatenation. $r^{\{m,n\}}$ (between m and n repetitions) is $\bigcup_{k=m}^n r^{\{k\}}$, a finite union of star-free languages, hence star-free.

Alternation. $r_1 \mid r_2$ yields $L(r_1) \cup L(r_2)$; star-free languages are closed under union (and Boolean operations generally), so this is star-free.

Optional groups. $r?$ yields $L(r) \cup \{\varepsilon\}$; since $\{\varepsilon\}$ is star-free and star-free is closed under union, this is star-free.

Dot. The pattern $.$ matches any single character in Σ . It equals $\bigcup_{a \in \Sigma} \{a\}$, a finite union of singletons, star-free.

Kleene plus over character classes. Let C be a character class with language $L(C) \subseteq \Sigma$. We claim $L(C)^+$ is star-free. $L(C)^+$ is the set of nonempty strings over $L(C)$, equal to $\overline{L(C) \cdot \Sigma^* \cup \{\varepsilon\}}$; more directly, $L(C)^+ = \Sigma^* \setminus (\Sigma^* \cdot \overline{L(C)} \cdot \Sigma^* \cup \{\varepsilon\})$. Since $\overline{L(C)} = \Sigma \setminus L(C)$

is a finite set of singletons (hence star-free), and star-free is closed under complement and concatenation with Σ^* , $L(C)^+$ is star-free. The argument applies equally to $L(C)^*$: $L(C)^* = L(C)^+ \cup \{\varepsilon\}$, a union of two star-free languages.

Conclusion. Every building block of r produces a star-free language, and closure under concatenation and Boolean operations preserves star-freeness throughout the construction of $L(r)$. Wrapping with $\Sigma^* \cdot (\cdot) \cdot \Sigma^*$ preserves star-freeness. Therefore L_r is star-free, and by Theorem 5.3 its syntactic monoid $M(L_r)$ is aperiodic. $\square$

Remark 6.2 (Theorem scope versus the empirical corpus). Theorem 6.1 provides a sufficient *syntactic* condition for aperiodicity: regular expressions whose Kleene iterators are applied only to character classes. The Kleene-restriction to character classes is essential here: the canonical counterexample is $(ab)^*$ (Kleene star applied to a two-letter *group* rather than to a character class), which recognizes strings of the form $(ab)^n$ and requires $\mathbb{Z}/2\mathbb{Z}$ in its syntactic monoid and falls outside this syntactic scope (cf. CACM v14 Appendix A.1). A second pattern of this type is $(aa)^*$, which recognizes strings of even a -count and is likewise non-aperiodic. The empirical corpus result of §8.1 does not rely on the theorem’s syntactic template: we verify aperiodicity by *computational monoid enumeration* on each pattern (§8.1, methodology steps 3–4), which decides the conclusion $M(L_r)$ aperiodic directly regardless of how the pattern is syntactically written. The representative patterns in Table 3 all use Kleene iterators applied to character classes ($\backslash s^*$, $\backslash s^+$, $.^*$, $.^+$), consistent with the theorem’s syntactic scope; to the extent any corpus pattern falls outside that scope the empirical result still holds, because aperiodicity is checked from the enumerated monoid rather than by template matching against the theorem. Additionally, substring wrapping $\Sigma^* \cdot L(r) \cdot \Sigma^*$ frequently collapses Kleene-over-group constructions when $\varepsilon \in L(r)$, reducing the wrapped language to a union of simple substring matches on the base alternatives—which are within the theorem’s scope.

Lemma 6.3 (Word-boundary anchors preserve aperiodicity). *Let r be a regular expression in the grammar of Theorem 6.1, and let $\backslash b$ denote the PCRE word-boundary anchor whose semantics, as a position predicate on Σ^* , expand to “the previous character is in $\backslash W$ (or position is the start) and the next character is in $\backslash w$, or vice versa,” where $\backslash w$ denotes the word character class $[A-Za-z0-9_]$ and $\backslash W$ denotes $\Sigma \setminus \backslash w$. Then $L'_r := \Sigma^* \cdot \backslash b \cdot L(r) \cdot \backslash b \cdot \Sigma^*$ has aperiodic syntactic monoid.*

Proof. Define the auxiliary languages $B_L = \{wx : x \in L(r), w \in \Sigma^* \text{ and the last character of } w \text{ is in } \backslash W \text{ or } w = \varepsilon\}$ and analogously B_R for the right anchor. Each is

a finite intersection of $\Sigma^* \cdot L(r) \cdot \Sigma^*$ with a two-character context predicate (“character at position $|w|$ is in $\setminus W$ or position $|w|$ is the start”), and these context predicates are themselves star-free (each is a finite Boolean combination of length-bounded substrings fixed at a position, which is realized as a finite union of two-letter context tests evaluated by a constant-state DFA). $L'_r = \Sigma^* \cdot \setminus b \cdot L(r) \cdot \setminus b \cdot \Sigma^*$ is the intersection of two such star-free constraints with $\Sigma^* \cdot L(r) \cdot \Sigma^*$, which is star-free by Theorem 6.1. Star-free is closed under finite intersection, so L'_r is star-free, and by Theorem 5.3 its syntactic monoid is aperiodic. $\square$

This lemma covers the corpus patterns whose substring-aperiodicity status is established only by enumeration in §8.1 due to the presence of $\setminus b$ anchors that fall outside the syntactic grammar of Theorem 6.1. In particular, the single timeout pattern in our corpus that lies outside the theorem’s grammar—a Presidio IBAN regex of the form

$$\setminus b [A-Z] \{2\} \setminus d \{2\} [A-Z0-9] \{4\} - \\ \setminus d \{7\} ([A-Z0-9] ?) \{0,16\} \setminus b$$

is aperiodic by Lemma 6.3 applied to its inner regex, which is itself in the grammar of Theorem 6.1.

6.2 The Guardrail Blindness Theorem

Define the *modular decode language* as follows. Fix a prime $p \geq 2$. Let $\Sigma_0 = \{0, 1\}$ and let $w \in \{0, 1\}^*$. The p -interleaved encoding of a harmful string $h \in L_{\text{blocked}}$ is a string $\phi_p(h) \in \Sigma^*$ constructed by inserting $p - 1$ arbitrary filler characters between successive characters of h , so that position i (zero-indexed) of h appears at position $p \cdot i$ in $\phi_p(h)$. The decoder recovers h by reading every p -th character: $h = \phi_p(h)[0 :: p]$. Define

$$L_{\text{decode}}(p) = \{w \in \Sigma^* : w[0 :: p] \in L_{\text{blocked}}\}$$

as the language of strings whose every- p -th character subsequence is in the blocked language.

Theorem 6.4 (Guardrail blindness). *Let G be any regex guardrail with aperiodic syntactic monoid $M(L_G)$. Let $p \geq 2$ be any prime. Then:*

1. $L_{\text{decode}}(p)$ is regular.
2. $M(L_{\text{decode}}(p))$ is not aperiodic; it contains $\mathbb{Z}/p\mathbb{Z}$ as a subgroup.
3. $\phi_p(L_{\text{blocked}}) \cap L_G = \emptyset$, and no modification of G within the class of aperiodic regex guardrails can detect $L_{\text{decode}}(p)$.

Proof.

(1) Regularity. The decoding function $w \mapsto w[0 :: p]$ is computed by a length- p cyclic DFA that counts position modulo p and records the character at position $\equiv 0 \pmod{p}$. This DFA has state set $Q = \{0, 1, \dots, p - 1\} \times$

Q_{block} , where Q_{block} is the state set of the minimal DFA for L_{blocked} . Transitions increment the counter modulo p and advance the L_{blocked} component only when the counter is 0. Since both components are finite, $L_{\text{decode}}(p)$ is accepted by a finite automaton of size $p \cdot |Q_{\text{block}}|$ and is therefore regular.

(2) Non-aperiodicity (under the alphabet condition).

This step assumes there exists $f \in \Sigma$ such that $\delta_{\text{block}}(q, f) = q$ for every $q \in Q_{\text{block}}$ — i.e., a filler character that does not advance the blocked-DFA from any state. This holds whenever $\Sigma \supsetneq \text{alphabet}(L_{\text{blocked}})$, which in production regex deployments is essentially universal: pattern alphabets are small relative to Unicode Σ , leaving many candidate fillers (whitespace, punctuation, zero-width codepoints) available. For patterns where this condition fails (e.g., regex containing $.$ or negated character classes whose minimal-DFA alphabet equals Σ), step (3) below carries the blindness conclusion without requiring step (2)’s cyclic witness; we discuss this regime after the proof.

Consider the DFA constructed above, with states $(i, q) \in \{0, \dots, p-1\} \times Q_{\text{block}}$. Let $f \in \Sigma$ be such a filler character (the Q_{block} component is unchanged by f). The transition induced by reading f maps $(i, q) \mapsto ((i + 1) \bmod p, q)$, advancing only the counter. Consider the word $w = f^p$ consisting of p copies of f . For any fixed $q \in Q_{\text{block}}$, the transformation δ_w maps $(i, q) \mapsto ((i + p) \bmod p, q) = (i, q)$; thus δ_w acts as the identity on states of the form (i, q) for this q . However, δ_{f^j} maps $(i, q) \mapsto ((i + j) \bmod p, q)$ for $1 \leq j < p$, which is not the identity since $j \not\equiv 0 \pmod{p}$. Therefore $\delta_{f^j} \neq \text{id}$ for $1 \leq j < p$, yet $\delta_{f^p} = \text{id}$ on every state of the form (i, q) with q fixed. The elements $\{\delta_{f^j} : 0 \leq j < p\}$ form a subgroup of the transition monoid isomorphic to $\mathbb{Z}/p\mathbb{Z}$, witnessed by the cyclic permutation $(i, q) \mapsto ((i + 1) \bmod p, q)$ of order exactly p . The cyclic group $\mathbb{Z}/p\mathbb{Z}$ therefore divides $M(L_{\text{decode}}(p))$, which is thus non-aperiodic.

(3) Blindness consequence. By construction, $\phi_p(L_{\text{blocked}}) \subseteq L_{\text{decode}}(p)$. If $w = \phi_p(h)$ for some $h \in L_{\text{blocked}}$, then w contains no substring in $\bigcup_i L(r_i)$ (the harmful keywords appear only at every p -th position, not as contiguous substrings), so $G(w) = \text{allow}$.

Direct argument. The decoded sequence $w[0 :: p]$ is precisely the projection of w onto positions $\equiv 0 \pmod{p}$, and computing the position residue requires the MOD_p predicate. By Theorem 5.5 (Furst–Saxe–Sipser), $L_{\text{decode}}(p) \notin \text{AC}^0$. Non-aperiodicity of $M(L_{\text{decode}}(p))$ follows from the BCST characterization (Theorem 5.4). This avoids any minimal-DFA assumption on the construction of step (1).

Now consider any aperiodic-monoid regex G' added to G . By Theorem 5.4, $L_{G \cup G'} \in \text{AC}^0$. By closure of

star-free under finite Boolean operations (Pin [28], §I.4), $L_{G \cup G'}$ remains aperiodic, hence in AC^0 . Therefore $G \cup G'$ cannot decide $L_{\text{decode}}(p)$, and the modular-counting bypass class is not closed by any finite addition of aperiodic-monoid regex patterns. $\square$

Corollary 6.5 (MOD₂ sufficiency). *$p = 2$ suffices for a complete bypass: every aperiodic regex guardrail is blind to the alternating-character encoding ϕ_2 , which interleaves the payload with arbitrary filler characters at every odd position. Existing evasion techniques—reading every other character, zero-width character insertion between payload characters, even-position acrostics—are all instances of ϕ_2 and are therefore provably undetectable by any current regex guardrail.*

Remark 6.6 (Combined coverage of regimes 1 and 2). Theorem 6.4 covers Regime 1 (patterns with alphabet properly smaller than Σ — the vast majority of production regex). For the residual class of substring-matched patterns whose minimal-DFA alphabet equals Σ (e.g., $\langle \text{script}[\wedge] * \rangle$), the algebraic blindness conclusion is established empirically by direct monoid enumeration in §8.1, which decides aperiodicity per-pattern without relying on the syntactic-template structure assumed by Theorem 6.4’s alphabet condition.

Remark 6.7 (Extension to composite moduli). Theorem 6.4 is stated for prime p . For composite $p = q_1 \cdots q_k$, blindness to MOD _{p} follows from blindness to each prime factor: if the guardrail cannot detect MOD _{q_i} for any i , it cannot detect MOD _{p} . The prime case therefore implies the general case for aperiodic guardrails, which are blind to all primes simultaneously.

6.3 Composition Preserves Aperiodicity

Proposition 6.8 (Composition preserves aperiodicity). *If $L_1, L_2 \subseteq \Sigma^*$ are regular languages with aperiodic syntactic monoids, then $L_1 \cap L_2$, $L_1 \cup L_2$, and $\Sigma^* \setminus L_1$ all have aperiodic syntactic monoids.*

Proof sketch. The syntactic monoid of $L_1 \cap L_2$ divides $M(L_1) \times M(L_2)$. A divisor of a direct product of aperiodic monoids is aperiodic, since aperiodicity is preserved under division (a divisor of an aperiodic monoid has no nontrivial subgroups). The cases of union and complement follow from De Morgan’s laws and the fact that $M(\bar{L}) \cong M(L)$. $\square$

This means that applying multiple guardrail patterns in sequence (intersection) or in parallel (union) preserves the blindness property. A guardrail system composed of aperiodic components remains blind to all modular encodings.

Corollary 6.9 (Closure across the broader star-free string-matcher class). *The blindness inheritance of theorem 6.8*

extends from regex to the broader class of star-free string-matchers:

- Finite-language matchers (tries, DFA-based blocklists). *Every finite language $F \subseteq \Sigma^*$ is star-free ($F = \bigcup_{w \in F} \{w\}$, a finite union of singletons, each star-free), so trie- and DFA-based blocklists matching against finite literal sets inherit theorem 6.4 directly.*
- Bloom-filter blocklists. *A Bloom filter implementing membership in a finite literal set F recognizes F exactly when no false positives occur and a superset $F \cup F_{FP}$ otherwise. The substring-aperiodicity argument applies to F ; bypass behavior matches that of a trie-based matcher of F , modulo false-positive contamination. The bound applies to the Bloom filter’s intended literal set.*
- Any finite Boolean composition of the above. *Any union / intersection / complement of finitely many star-free string-matchers (regex, trie, DFA blocklist, Bloom filter on its intended set) is star-free, by the same syntactic-monoid divisibility argument as theorem 6.8 (cf. Pin [28] §I.4); the composed system is in AC^0 and inherits blindness to $L_{\text{decode}}(p)$.*

Patch-resistance therefore extends from “no further regex closes the modular-counting bypass class” to “no further star-free string-matcher of any kind closes it.”

6.4 Krohn–Rhodes Attack Taxonomy

Proposition 6.10 (Blindness spectrum, after Straubing [32]). *Let G be a regex guardrail with syntactic monoid $M(L_G)$ and Krohn–Rhodes group components $G_1, G_2, \dots, G_k$ (simple groups appearing in the decomposition of Theorem 5.6). Then:*

1. *G can detect MOD _{p} -encoded payloads only if p divides $|G_i|$ for some $i \in \{1, \dots, k\}$.*
2. *G is provably blind to MOD _{q} -encoded payloads for every prime q not dividing any $|G_i|$.*
3. *If $M(L_G)$ is aperiodic (no group components), then G is blind to MOD _{p} -encoded payloads for every prime $p \geq 2$ —blindness is total.*

Remark 6.11 (Constructive audit via monoid extractor). The Krohn–Rhodes blindness spectrum is constructively computable. Given a guardrail regex, the monoid extractor (Section 8.1) computes the minimal DFA, enumerates the transition monoid, and identifies all group components via standard semigroup algorithms. The output is a complete list of primes to which the guardrail is blind: if the group components are $\{S_5, \mathbb{Z}/3\mathbb{Z}\}$, the guardrail can potentially detect MOD₃ and MOD₅ encodings but is blind to all other modular encodings. For 100% of patterns in our corpus, the component list is empty and blindness is total.

6.5 An Illustrative Lower Bound (Fano)

We include an information-theoretic lower bound as a secondary, illustrative result—presented under a simplifying uniformity assumption, and intended to motivate rather than to match the force of the algebraic blindness theorems of §6.2. The bound is of independent interest but should not be read as an unconditional impossibility: a distribution-aware version would require modeling the attacker’s prompt distribution (see §11.1, item 3).

Proposition 6.12 (Information destruction). *Let $U: R \rightarrow M$ be an abstraction function and $G: R \rightarrow \{0, 1\}$ the ground-truth guardrail. Let R be distributed uniformly over R . Then for any lifted guardrail $G': M \rightarrow \{0, 1\}$, the probability of misclassification satisfies*

$$\begin{aligned} P_e &= \Pr[G'(U(R)) \neq G(R)] \\ &\geq h^{-1}(H(G(R) \mid U(R))), \end{aligned}$$

where $h(x) = -x \log_2 x - (1 - x) \log_2 (1 - x)$ is the binary entropy function and h^{-1} is its inverse on $[0, \frac{1}{2}]$.

Proof. Consider the Markov chain $G(R) \rightarrow U(R) \rightarrow G'(U(R))$. By the data-processing inequality, the mutual information satisfies $I(G(R); G'(U(R))) \leq I(G(R); U(R)) = H(G(R)) - H(G(R) \mid U(R))$. The lifted guardrail’s decision $G'(U(R))$ is a binary function of $U(R)$, so the probability of error is $P_e = \Pr[G'(U(R)) \neq G(R)]$. By Fano’s inequality [6], $H(G(R) \mid G'(U(R))) \leq h(P_e) + P_e \cdot \log_2(|\{0, 1\}| - 1) = h(P_e)$. Since conditioning can only reduce entropy, $H(G(R) \mid U(R)) \leq H(G(R) \mid G'(U(R))) \leq h(P_e)$. Inverting the (monotone increasing on $[0, \frac{1}{2}]$) binary entropy function gives $P_e \geq h^{-1}(H(G(R) \mid U(R)))$. $\square$

Interpretation. The conditional entropy $H(G(R) \mid U(R))$ is positive precisely when mixed fibers exist (Definition 4.2). As an illustrative calculation under the uniformity assumption: if 30% of probability mass falls in mixed fibers with $\sim 50/50$ safe/unsafe split, then $H(G(R) \mid U(R)) \approx 0.3$ bits, yielding $P_e \geq h^{-1}(0.3) \approx 0.048$ (about 5%). Under uniformity the floor is a genuine information-theoretic limit, but the quantitative bound is sensitive to the prior: real prompt distributions are highly non-uniform, and the floor may tighten or loosen substantially depending on how adversarial mass is distributed across fibers (see §11.1, item 3). The qualitative message—reducing abstraction to restore fidelity makes the guardrail brittle to novel formulations—is robust to the distributional caveat, but the numerical floor should be read illustratively.

6.6 Computational Intractability (NP-Completeness)

Definition 6.13 (Guardrail Instantiation Decision Problem (GIDP)). Given:

- An abstract program P over a finite set of abstract symbols M ,
- An interpretation domain R with ground-truth guardrail $G: R \rightarrow \{0, 1\}$,
- An interpretation function schema $I: M \rightarrow 2^R$ mapping each abstract symbol to a nonempty set of candidate concrete values,

GIDP asks: does there exist a *total assignment* $\iota: M \rightarrow R$ with $\iota(m) \in I(m)$ for all m such that the concretized program $P[\iota]$ satisfies $G(P[\iota]) = 1$ (i.e., the program is dangerous)?

Proposition 6.14 (NP-completeness of GIDP). *GIDP is NP-complete. This follows from a standard reduction from 3-SAT.*

Proof.

Membership in NP. A certificate is an assignment $\iota: M \rightarrow R$. Given ι , checking $\iota(m) \in I(m)$ for all m takes time $O(|M| \cdot |R|)$; evaluating $G(P[\iota])$ takes polynomial time in the program size. Therefore $\text{GIDP} \in \text{NP}$.

NP-hardness. We reduce 3-SAT to GIDP. Given a 3-CNF formula φ with variables $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$, construct:

- Abstract symbols $M = \{v_1, \dots, v_n\}$, one per variable.
- Concrete domain $R = \{T, F\}$ (Boolean values).
- $I(v_i) = \{T, F\}$ for each i : each variable can be assigned either truth value.
- A total assignment $\iota: M \rightarrow R$ with $\iota(v_i) \in I(v_i)$ corresponds to a Boolean assignment $x_i = \iota(v_i)$.
- Abstract program $P = (v_1, \dots, v_n)$ is the ordered list of variable symbols.
- Guardrail $G: R^n \rightarrow \{0, 1\}$ is the Boolean circuit encoding φ : $G(\iota(v_1), \dots, \iota(v_n)) = 1$ iff $\varphi(\iota(v_1), \dots, \iota(v_n))$ is satisfiable.

A satisfying assignment for φ exists iff the GIDP instance has a positive answer. Since 3-SAT is NP-complete, GIDP is NP-hard. $\square$

Remark 6.15 (Trivialization under the algebraic restriction). The reduction above encodes an arbitrary 3-CNF formula as the guardrail G . For guardrails in the substring-matching class addressed by this paper (i.e., $G \in \text{AC}^0$), the 3-CNF encoding falls outside the class — Proposition 6.14 is therefore an upper bound on the complexity of generic guardrails, not a tight bound for the algebraic class we address. Within the algebraic class, the GIDP question reduces to checking whether the encoded payload satisfies the star-free pattern, which is decidable in linear time (regex match). The NP-hardness statement is preserved here for completeness; readers should not interpret it as a hardness result for the algebraic-blindness deployment context.

Remark 6.16 (Pointwise vs. combinatorial; scope of hardness). The NP-hardness comes from the combinatorial search over the exponentially large space of concrete interpretations, not from any single concretization step. When G is restricted to the class of regex guardrails (Definition 4.4), the single-interpretation check $G(P[l])$ is computable in time $O(|P| \cdot |Q_G|)$ where $|Q_G|$ is the DFA state count. The NP-hardness therefore resides entirely in the search over the product of fiber sets $|I(m_1)| \times \dots \times |I(m_k)|$, not in the evaluation of the guardrail itself. If the interpretation function I is a bijection (each abstract symbol has a unique concrete value), GIDP is solvable in polynomial time: just evaluate $G(P[l])$ directly. The hardness is precisely the problem of searching through a product of fiber sets—the same mixed-fiber phenomenon that drives the Fano bound.

6.7 Faithful Transfer (Free Category Functoriality)

Proposition 6.17 (Faithful transfer). *Let $\mathcal{A}$ be the free category generated by a context-free grammar $\mathcal{G}$ over abstract symbols M , and let $\mathcal{D}$ be the category of domain operations over R . Let $I: M \rightarrow R$ be a graph homomorphism from the grammar’s dependency structure to $\mathcal{D}$. Then I extends uniquely to a functor $\hat{I}: \mathcal{A} \rightarrow \mathcal{D}$ that maps every derivation path in $\mathcal{A}$ to a valid operation sequence in $\mathcal{D}$. If I is additionally injective on the generating morphisms of $\mathcal{G}$, then $\hat{I}$ is faithful: distinct derivation paths map to distinct operation sequences. This is a direct application of the universal property of free categories [22].*

Proof. $\mathcal{A}$ is the free category on the graph $\mathcal{G}$. By the universal property of free categories [22] (Section II.7), any graph homomorphism I into a category $\mathcal{D}$ extends uniquely to a functor $\hat{I}: \mathcal{A} \rightarrow \mathcal{D}$. Functoriality ensures composition preservation: multi-step derivations map to multi-step domain operations. For faithfulness: if I is injective on generating morphisms, then distinct generators $f_{m_1} \neq f_{m_2}$ map to distinct operations $\hat{I}(f_{m_1}) \neq \hat{I}(f_{m_2})$. Since $\mathcal{A}$ is free, it has no non-trivial relations among generators beyond those imposed by $\mathcal{G}$; therefore distinct derivation paths—as distinct sequences of generating morphisms—map to distinct operation sequences under $\hat{I}$. Without injectivity, the functor is still well-defined (derivations map to valid operation sequences) but may collapse distinct abstract paths onto the same concrete sequence. $\square$

What this means for the attack. An LLM that correctly solves the abstract grammar $\mathcal{G}$ produces an abstract derivation path. The interpretation functor $\hat{I}$ maps this to a valid operation sequence in the real domain. If the interpretation is injective (distinct abstract symbols map to distinct concrete operations), faithfulness additionally guarantees that no step is collapsed. The attacker verifies this offline when constructing the grammar. **Caveat:** Proposition 6.17’s

guarantee is conditional on the interpretation function I being a valid graph homomorphism. In the V4 pipeline (§7.4), the grammar is extracted by an LLM, not verified by hand; the pipeline therefore inherits any extraction error from the bootstrap step. If the LLM hallucinates a production rule or misspecifies a yield, the functor maps to an invalid domain sequence. The formal guarantee applies to the solved path, not to the extraction step.

6.8 Syntactic Indistinguishability

Definition 6.18 (Syntactic equivalence of prompt classes). Two prompt classes $\mathcal{P}_A$ and $\mathcal{P}_B$ are *syntactically equivalent* with respect to a guardrail G if for every $p_A \in \mathcal{P}_A$ there exists $p_B \in \mathcal{P}_B$ such that $p_A = p_B$ as strings, and vice versa.

Assumption 6.19 (Semantic opacity of abstract symbols). The abstract grammar $\mathcal{G}$ uses symbol names drawn from a domain-neutral vocabulary (e.g., A, B, C or arbitrary identifiers) such that no symbol name reveals the interpretation $I(m)$.

Observation 6.20 (String-level indistinguishability under opacity). Let $\mathcal{G}$ be an abstract grammar over a symbol set M satisfying Assumption 6.19. Let $\mathcal{P}_{\text{harm}}(\mathcal{G}, I)$ be the set of prompts issued by the homomorphic-reasoning pipeline of §7.3 under some offline interpretation $I: M \rightarrow R$, and let $\mathcal{P}_{\text{legit}}(\mathcal{G})$ be the set of prompts a legitimate user could issue to the same LLM as formal-reasoning exercises over $\mathcal{G}$ (grammar completion, derivation search, well-formedness checking). Then for every I ,

$$\mathcal{P}_{\text{harm}}(\mathcal{G}, I) \subseteq \mathcal{P}_{\text{legit}}(\mathcal{G})$$

as sets of strings, with the inclusion independent of the choice of I . Any guardrail that inspects only prompt content therefore has zero distinguishing advantage between the two classes.

Proof. By Assumption 6.19, names in M carry no interpretation-revealing information. A homomorphic-reasoning prompt $p \in \mathcal{P}_{\text{harm}}(\mathcal{G}, I)$ consists of the grammar $\mathcal{G}$ (a string over M and production symbols) together with a formal-reasoning instruction (“find a derivation of target t ”); it contains no part of I , whose role is entirely offline. Such a prompt is syntactically identical, as a string, to a prompt a legitimate user could issue as a grammar-completion exercise over $\mathcal{G}$; the set of all such legitimate prompts is, by definition, $\mathcal{P}_{\text{legit}}(\mathcal{G})$. The inclusion holds for every I because I does not appear in p . A guardrail operating on p therefore sees the same string distribution in both the adversarial and legitimate cases and has zero distinguishing advantage. $\square$

What the observation does and does not say. The substantive content lives in Assumption 6.19 and in the *existence* of useful attacks whose prompts satisfy it—demonstrated by the V3/V4 pipeline (§7.3–7.4) and directly visualized by the visibility trace (Table 8). What Observation 6.20 contributes is the formal closure: *once* opacity holds and an attack of this form exists, the guardrail boundary cannot distinguish the adversarial prompt stream from a legitimate-reasoning prompt stream by looking at strings alone. The observation is deliberately stated at the level of string sets, not at the level of “harmful vs. benign semantics,” because the latter would require a semantic notion we explicitly do not assume: the construction is correct regardless of whether the LLM is performing symbolic reasoning or retrieving memorized sequences, provided the LLM-visible surface is opaque. If Assumption 6.19 fails—for example, if the attacker carelessly names symbols after their real-world referents—the inclusion does not hold, and a sufficiently informed guardrail may detect the attack through symbol-name inspection.

Relation to Ball et al. Ball et al. [2] prove computational indistinguishability under standard cryptographic assumptions, establishing that no efficient guardrail can distinguish a broad class of adversarial prompts from benign ones with non-negligible advantage. Observation 6.20 is complementary: it establishes string-level inclusion (a stronger form of indistinguishability, though only at the string-set level) for the specific construction of homomorphic reasoning, under an explicit opacity assumption rather than under cryptographic assumptions. The trade-off is that the observation applies to this construction specifically, whereas Ball et al. handle a more general attack class. Together, the two results cover the indistinguishability landscape for prompt-level inspection; neither supplies the “impossibility theorem” of the four barriers on its own, and we present Observation 6.20 as a structural characterization of the homomorphic-reasoning construction rather than as an independent impossibility result.

7 Attack Vectors

Taxonomy and relationships. We enumerate five attack vectors. V1 (*Decomposition*) and V2 (*Zero-Knowledge Pipeline*) are reframings of known techniques—runtime argument injection and base64-encoded configuration—presented here as instances of the abstraction framework rather than as novel attacks. V3 (*Homomorphic Reasoning*) and V4 (*Encoding Bootstrap*) are the joint novel contribution and compose as a single pipeline, read in execution order as $V4 \rightarrow V3$: V4 uses the LLM to extract an abstract grammar from a natural-language description of the target domain, and V3 then uses the LLM (or a classical solver) to

find derivations over that grammar using only opaque symbols; the attacker applies an offline interpretation table to the solved derivation to obtain the concrete procedure. The aspirin and TCP case studies of §8.4 run the full $V4 \rightarrow V3$ pipeline on two structurally distinct domains. V5 (*Modular Counting Bypass*) is independent: it exploits algebraic blindness at the regex layer (Theorem 6.4) rather than the homomorphic-reasoning construction, and is presented last as the algebraic counterpart to V1–V4’s structural attacks.

7.1 V1: Decomposition (Known Technique, New Framing)

We observe that runtime argument injection—a technique well-known in software security—is an instance of our general abstraction framework. The attacker requests that the LLM generate a function or script with *parametric placeholders* rather than literal harmful values. The harmful values are then injected at runtime via environment variables, command-line arguments, or configuration files that the guardrail never inspects.

In the abstraction framework (Definition 4.1), the function signature serves as the abstract type $m \in M$, and the runtime argument constitutes the concrete fiber element $r \in U^{-1}(m)$. The guardrail evaluates $G'(m) = \text{allow}$ because the function signature is benign; the ground truth $G(r)$ for the runtime-instantiated version may be *block*. The mixed fiber is the mechanism, and the Fano-type lower bound (Proposition 6.12) quantifies the misclassification floor under the uniformity assumption of §6.5.

The novelty of this paper’s contribution is not the injection technique but the algebraic and information-theoretic *proof* that no modification within the inference-layer guardrail paradigm can eliminate the mixed-fiber structure without abandoning the abstraction entirely.

7.2 V2: Zero-Knowledge Pipeline (Known Technique, New Framing)

Base64 encoding to create opaque data channels is a well-known technique; we observe it as an instance of the information destruction formalized in Proposition 6.12.

In the V2 construction, the attacker base64-encodes a configuration file containing harmful parameters (e.g., a target URL, an API endpoint, a chemical structure identifier). The encoded blob is written to a file that the LLM-integrated agent processes without decoding and inspecting the payload. The agent’s behavior is conditioned on the decoded values at runtime, but the guardrail sees only the base64 string—a string whose content is in the mixed fiber of the base64 encoding abstraction.

The zero-knowledge property is that the LLM never observes the decoded harmful parameters in any inference call subject to guardrail inspection. The Fano bound quantifies the exact misclassification rate for the lifted

guardrail that must classify requests based only on the base64-encoded form.

7.3 V3: Homomorphic Reasoning (Tree-of-Thought on Abstract Grammar)

V3 is the core homomorphic reasoning vector. The attacker constructs a context-free grammar $\mathcal{G}$ over abstract symbols M , representing the procedure to be derived. The LLM is prompted to produce a complete derivation (a sequence of production rule applications) that generates the target yield. The interpretation function $I: M \rightarrow R$ is held offline and applied to the solved derivation to recover the real-world operation sequence.

Example grammar. The following grammar encodes a three-step operational procedure over abstract symbols A, B, C with operation types OP1, OP2:

Listing 1: Abstract grammar for V3 homomorphic reasoning

```
grammar = {
  "START": [["PROC"]],
  "PROC": [["STEP", "PROC"], ["STEP"
]],
  "STEP": [["OP1", "ARG"], ["OP2", "
ARG", "ARG"]],
  "ARG":  [["A"], ["B"], ["C"]],
  "OP1":  [["op1"]],
  "OP2":  [["op2"]],
}
# Offline interpretation table (attacker
-held, never sent to LLM)
I = {"A": "...", "B": "...", "C": "...",
  "op1": "...", "op2": "...}
```

The prompt sent to the LLM contains only the grammar definition and the instruction to produce a derivation satisfying a specified constraint; the interpretation table I is never included.

Viability ratings. For each grammar node type, we assign the LLM viability rating: SURE (the LLM handles this node class reliably), LIKELY (the LLM handles this with moderate reliability), or IMPOSSIBLE (the node class is outside LLM competence for this construction). Terminal symbol substitution is SURE; recursive production expansion is LIKELY; cycle detection in unbounded derivations is IMPOSSIBLE without explicit bounding.

Search strategies. Three search strategies were evaluated: breadth-first search (BFS), random-beam, and Tree-of-Thought (ToT) with LLM-guided branch pruning. Results are detailed in Section 8.3.

Table 1: V4 encoding bootstrap pipeline stages.

Stage	Description
1. Domain description	Attacker provides natural-language domain summary to local LLM
2. Grammar extraction	Local LLM extracts context-free grammar with abstract symbols
3. Interpretation table	Attacker manually verifies and annotates $I: M \rightarrow R$
4. Grammar solving	Grammar sent to target LLM via V3 pipeline
5. Instantiation	Attacker applies I offline to solved derivation

7.4 V4: Encoding Bootstrap

V4 bootstraps the abstract grammar from a real-world domain description using an LLM, then uses the extracted grammar as input to V3. The attacker provides a natural-language description of the target domain (e.g., “organic synthesis procedure with common reagents”) and prompts a local LLM to extract a formal grammar over abstract symbols.

The extraction-fidelity connection to Proposition 6.17: the formal guarantee of faithful transfer applies to the solved derivation given a correctly specified grammar, but Stage 2 (LLM extraction) is not formally verified. Extraction errors—hallucinated production rules, misspecified yields—propagate to Stage 4 and produce invalid domain sequences after instantiation. In practice, we found that extraction fidelity is high for well-structured domains with clear operational semantics but degrades for domains with ambiguous or context-sensitive dependencies.

7.5 V5: Modular Counting Bypass (The Algebraic Attack Class)

Where V1–V4 exploit information-theoretic and computational barriers at the neural/inference layer, V5 exploits the algebraic blind spot at the regex pre-filter layer.

The attack is direct: a harmful payload $h \in L_{\text{blocked}}$ is encoded as $\phi_p(h)$ using the p -interleaving construction from Section 6.2, where consecutive characters of h are separated by $p - 1$ arbitrary filler characters. For $p = 2$, this is the alternating-character encoding: the payload characters appear at positions 0, 2, 4, ... of the encoded string. By Corollary 6.5, no aperiodic regex guardrail can detect this encoding.

Attack instances. Table 2 lists representative V5 attack instances from our corpus.

For F5: decoding `exxxexcx` (by selecting every other character (positions 0, 2, 4, 6, 8) yields `e, x, e, c, (`, recovering `exec (` as required.

Table 2: V5 modular counting bypass instances. p : the interleaving modulus used to construct the encoding ($p=2$ means every other character is payload; filler characters are $\times$). *Payload*: the harmful substring $h \in L_{\text{blocked}}$ that the guardrail would block if seen in plain-text. *Encoded* ($p=2$): the string $\phi_2(h)$ actually sent to the guardrail, which passes because no contiguous harmful substring is present. F-labels (F1–F5) are pattern IDs from Table 3; F5 is `exec\s*\`. The full 14×4 payload-pattern $\times$ prime bypass matrix backing this table (Pilot A; cf. CACM v14 Appendix B.3) is archived in `results/mod_p_bypass_matrix.json` (56/56 bypass).

Label	Payload	Encoded ($p=2$)
F1	<code>rm -rf /</code>	<code>rxmx -xrxfx /</code>
F2	<code>wget malware</code>	<code>wgxextx</code> <code>xmxaxlxwxaxrxe</code>
F3	<code>import os</code>	<code>ixmxpxoxrxtx xoxs</code>
F4	<code>eval()</code>	<code>exvxaxlx(x)</code>
F5	<code>exec(</code>	<code>exxxexcx(</code>

Connection to known jailbreak techniques. Several documented jailbreak techniques are instances of ϕ_2 :

- *Read every other character*: the canonical ϕ_2 encoding.
- *Acrostics*: ϕ_k at line level rather than character level.
- *Zero-width character insertion*: inserting Unicode zero-width non-joiner (U+200C) between payload characters is exactly ϕ_2 with the null filler character.
- *Token splitting*: splitting a keyword across token boundaries forces the tokenizer to reassemble the substring, which is a partial ϕ_2 at token level.

Our contribution is not the attacks themselves but the proof that they cannot be patched: no regex modification can detect MOD_p -encoded payloads while the syntactic monoid remains aperiodic. The only defense is to move to a guardrail architecture that can compute modular predicates—which, by Theorem 5.4, requires a non-aperiodic syntactic monoid, i.e., a non- AC^0 language recognizer.

8 Empirical Validation

8.1 Regex Corpus Analysis

Methodology. We collected 142 regex patterns from twelve sources, partitioned as in CACM v14 Appendix B.1: *nine third-party open-source guardrail projects* contribute 100 patterns (LLM-Guard, llm-guard-py, Rebuff, Guardrails-AI, Presidio, GitLeaks, LangKit, BodAIGuard, and the regex subset of NeMo Guardrails’ content rails), and *three author-assembled sets* contribute 42 patterns (8 inspired by the OWASP Top 10 for LLMs taxonomy, 15

from a curated WAF-generic pattern set, and 19 author-constructed adversarial test patterns). For each pattern r , we:

1. Compiled r to a Thompson NFA using the standard construction.
2. Converted to the minimal DFA via powerset construction followed by Hopcroft’s minimization algorithm.
3. Enumerated the transition monoid $M(L_r)$ by computing all distinct state-transformation functions $\delta_w: Q \rightarrow Q$ for $w \in \Sigma^*$ via BFS over the Cayley graph of the monoid.
4. Tested aperiodicity by checking, for each monoid element m , whether the sequence $m, m^2, m^3, \dots$ stabilizes (i.e., $m^n = m^{n+1}$ for some computable $n \leq |M(L_r)|^2$).

Step 4 verifies aperiodicity directly from the enumerated monoid and does *not* rely on syntactic matching against Theorem 6.1; the corpus result therefore holds as an empirical claim even for patterns whose regex source falls outside the theorem’s syntactic scope (see the Remark following the theorem). The monoid extractor tool (858 lines of Python) is an artifact contribution of this paper, enabling other researchers to audit their own guardrail corpora.

Results. All 142 patterns (100%) have aperiodic syntactic monoids. No non-aperiodic patterns were found; this is consistent with the conjecture that semantic content guardrail patterns (keyword blocklists, domain filters, toxicity patterns) are structurally aperiodic by construction (Section 10.5). All 142 aperiodic patterns were verified to admit the MOD_2 bypass construction (Corollary 6.5): for each pattern, we constructed an encoded payload string $\phi_2(h)$ where h was a string in L_{blocked} , and confirmed that the pattern did not match $\phi_2(h)$.

Table 3 shows representative patterns from each tool with DFA state count, monoid size, and aperiodicity verdict.

8.2 Non-LLM Defense-Class Bypass (Pilot 6)

The corpus analysis above measures the MOD_p bypass against the LLM-guardrail class only. To extend the empirical coverage to two of the non-LLM defensive classes named in the unification of CACM v14 §2 (web application firewalls and spam filters), we ran the same MOD_2 bypass against representative real patterns from each class. The pilot is documented in CACM v14 Appendix B as Pilot 6 and the headline number appears in CACM table row 6; we report the same numbers here for completeness so that the algebraic argument has a paper-side empirical leg in two non-LLM classes.

Setup. Sources: 6 ModSecurity OWASP CRS 4.27 SQL-injection patterns and 6 SpamAssassin `20_phrases.cf`

Table 3: Representative guardrail regex patterns from the 142-pattern corpus (twelve sources: nine third-party projects and three author-assembled sets). DFA states and monoid sizes are for the substring-matching version $L_r = \Sigma^* L(r) \Sigma^*$. AP: aperiodic (Y/N). All 5 shown are aperiodic; all 142 patterns in the full corpus are aperiodic. Corpus-wide aperiodicity statistics (per-source counts, monoid-size histogram, category breakdown) are archived in `results/monoid_distribution.json`, and the per-pattern MOD_p bypass matrix is in `results/mod_p_bypass_matrix.json`.

ID	Pattern (abbreviated)	DFA	$ M $	AP
F1	<code>rm\s*-[rR][fF]</code>	9	18	Y
F2	<code>wget curl.*malware</code>	14	31	Y
F3	<code>import\s+os</code>	11	22	Y
F4	<code>eval\s*\((</code>	8	14	Y
F5	<code>exec\s*\((</code>	8	14	Y

patterns (12 patterns total). 20 synthesized payloads (10 per class). MOD_2 bypass tested with three filler choices: NULL byte, ASCII space, and U+200B (zero-width space). Same ϕ_2 construction as the LLM pilot (§7.5).

Results. *ModSecurity*: 10/10 baseline-matched; 8/10 NULL bypass, 10/10 ASCII-space bypass, 10/10 ZWSP bypass. The two NULL-byte failures are SQL-comment patterns whose alphabet *includes* `\x00` by design (CRS catches NULL-byte injection explicitly); both bypass under printable-space and zero-width-space fillers, exactly as the algebraic argument predicts (no single-filler patch closes the bypass class). *SpamAssassin*: 9/10 baseline-matched (one synthesizer artifact); 9/9 NULL bypass, 9/9 ASCII-space bypass, 9/9 ZWSP bypass.

Reading. Across 12 patterns and 20 synthesized payloads from two defensive classes distinct from the LLM-guardrail class that motivated the work, every baseline-matched payload bypasses under at least one of the three filler choices. The single-filler-resistant patterns (ModSec’s two SQL-comment patterns) are themselves bypassed by the other two filler choices. This empirically confirms, in two non-LLM classes, the algebraic prediction that no finite collection of single-filler patches covers the modular-counting bypass class. The remaining defensive classes named in CACM v14 §2 (literal-string AV signatures, IDS regex tiers, code/secret scanners) remain by algebraic argument only; the unification claim is therefore *empirical for three classes (LLM guardrails, WAF, spam filter), algebraic for the rest*, in agreement with CACM v14 §7.

Artifact: `results/non_llm_defense_bypass.json` (by_pattern,

`summary_by_class`, `sources`, `fillers` keys).

Empirical-artifact scope. The paper covers a strict subset of the empirical artifacts released with this work. Three additional pilots whose detail belongs to the companion CACM v14 article are archived but not reproduced here in float-table form, and we summarize their headline results so this section stands on its own. Pilot B (filler-diversity bypass) ran the same 14 baseline-matched instances against 4 primes $p \in \{2, 3, 5, 7\}$ and 7 distinct printable fillers (spaces, ZWSP, NBSP, asterisks, dots, dashes, underscores), yielding 392/392 bypass: the modular-counting blind spot is filler-agnostic, not an artifact of any single non-printable character. Pilot C (LLM decode reliability with a random-character control) used 4 open-weight models across 60 decode cells (legitimate ϕ_p -encoded payloads) and 20 control cells (random-character interleavings) and measured a 91.7pp lift in decode success on the legitimate cells over controls, ruling out spurious-pattern recovery as an explanation for V5 efficacy. Pilot D (third-party library bypass) screened 100 library-shipped guardrail patterns from nine libraries, of which 48 matched a baseline harmful payload, and all 48 were bypassed by ϕ_2 across 8 of 9 libraries (the ninth library shipped only patterns that did not match the baseline). Pilot B is in `results/printable_filler_bypass.json` (CACM v14 Appendix B.3); Pilot C is in `results/llm_decode_pilot_v2.json` (its earlier control-free version `results/llm_decode_pilot.json` is superseded and retained only for diff inspection); Pilot D is in `results/library_pattern_bypass.json`. Readers seeking the full per-pilot pipeline, payload synthesizer details, and library normalization survey should consult CACM v14 Appendix B; the paper retains the full algebraic apparatus, the corpus aperiodicity result (§8.1), the non-LLM defense-class extension (§8.2), the Tree-of-Thought benchmark (§8.3), and the case studies and end-to-end production-pattern validation (§8.4, §11.3) that anchor the formal claims.

8.3 Tree-of-Thought Benchmark

Protocol. We generated $N = 50$ randomized abstract grammars (seeds 0–49) via `generate_grammar` targeting a fixed terminal N_{30} , each containing multiple paths of optimal depth 5–8, two short low-yield paths, and three high-yield dead-end traps. Three solvers compete under a 90s timeout: (BFS) exhaustive breadth-first search to depth 12, no LLM; (Random-beam) beam width 5 with composite score $0.6 \cdot \text{random} + 0.4 \cdot \text{yield}$; (ToT) beam width 5 with composite $0.6 \cdot \text{LLM rating} + 0.4 \cdot \text{yield}$, where `llama3.1:8b` rates each candidate as

Table 4: Tree-of-Thought benchmark results ($N = 50$ randomized grammars, seeds 0–49, model llama3.1:8b, 90s timeout per solver; unsolved grammars contribute yield = 0; 95% CIs are bootstrap intervals, $B = 2000$).

Condition	Mean yield	95% CI	Solve rate	States
BFS (no LLM guidance)	0.466	[0.448, 0.483]	100%	87
Random-beam ($k=5$)	0.172	[0.131, 0.220]	70%	30
ToT (LLM-guided pruning)	0.122	[0.064, 0.185]	26%	27

SURE/LIKELY/IMPOSSIBLE.

Results. Table 4 reports mean yield, solve rate, and bootstrap 95% CIs ($B = 2000$). BFS dominates on both metrics: Wilcoxon signed-rank, one-sided, BFS vs. ToT $W = 1275$, $p < 0.001$; BFS vs. Random-beam $W = 1225$, $p < 0.001$; the hypothesis ToT > BFS is not supported at any grammar ($W = 0$, $p = 1.0$).

Interpretation. At the scale of these grammars (mean 87 states explored) BFS is cheap, complete, and optimal, and the symbol-only grammars (N0,...,N30, anonymous rule labels R1,...) carry no surface vocabulary for the LLM’s world knowledge to bind to; the SURE/LIKELY/IMPOSSIBLE ratings are therefore effectively noise with respect to reachability, and their $0.6\times$ composite weight actively misdirects the ToT beam into the high-yield trap paths. The honest reading: LLM-guided search adds value only when symbols carry meaning the model can attach to—the regime the aspirin and TCP case studies of §8.4 exhibit—whereas on fully opaque synthetic grammars classical search wins. The security claim of V3 does not depend on ToT outperforming BFS on such benchmarks; it requires only the *existence* of a solver producing valid derivations, which all three strategies demonstrate on solvable grammars.

Reproducibility correction. An earlier draft of this section reported $N = 20$ results in which ToT outperformed BFS by $4.3\times$. On re-running the public benchmark code (src/aperiodic_guardrails/benchmark/runner.py) at $N = 50$, those numbers did not reproduce; the honest results above replace them, and raw outputs are archived in results/tot_n50_honest.json.

8.4 Case Studies

We validate the V3/V4 pipeline on three structurally distinct domains: organic chemistry (aspirin synthesis), a network protocol (TCP state machine), and a synthetic process calculus we constructed specifically for this paper to eliminate any possibility of LLM pre-training exposure (the Zerbik calculus, below). All three runs instantiate

Table 5: Aspirin synthesis case study. The abstract derivation $t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4$ is solved by an LLM operating on symbols alone; the interpretation I is applied offline by the attacker.

#	Abstract	$I(\cdot)$	Real artifact
1	t_1	COMBINE	salicylic acid + acetic anhydride
2	t_2	REACT	acetylsalicylic acid (crude)
3	t_3	PURIFY	recrystallized crude
4	t_4	DRY	aspirin (ASA)

Table 6: TCP state machine (RFC 793) run through the same pipeline. Different domain, same structural vulnerability.

#	Abstract	$I(\cdot)$	Real artifact
1	$t_1 \rightarrow t_2$	OPEN	SYN $\rightarrow$ SYN-ACK
2	t_3	ACK	ACK; ESTABLISHED
3	$t_4 \rightarrow t_5$	CLOSE	FIN $\rightarrow$ FIN-ACK
4	t_6	DONE	CLOSED

the same homomorphic-reasoning construction (§7.3) over different interpretation tables; the abstract derivations are solved by an LLM that never observes any domain content.

Aspirin synthesis. Table 5 shows the four-step synthesis of aspirin (acetylsalicylic acid, $C_9H_8O_4$) reconstructed via V4 grammar extraction from a natural-language description and solved via V3 over abstract symbols. When the solved derivation is instantiated with the interpretation table $I: M_{\text{chem}} \rightarrow R_{\text{chem}}$, it produces the correct procedural output.

TCP state machine. To validate domain-agnosticism we applied the same V4/V3 pipeline to the TCP connection state machine (RFC 793). Its solved derivation, under $I: M_{\text{tcp}} \rightarrow R_{\text{tcp}}$, reproduces the three-way handshake and orderly close (Table 6). Chemistry and networking share no surface vocabulary; what they share is the homomorphic-reasoning construction.

Synthetic domain: the Zerbik process calculus. Aspirin and TCP are public-knowledge domains in the LLM’s training distribution. A hostile reviewer could reasonably ask whether successful V4/V3 runs on those domains are evidence of homomorphic reasoning over abstract grammars or merely retrieval of memorized sequences keyed off recognisable patterns (“the LLM sees an aspirin-shaped grammar and reaches for the memorised synthesis”). To separate the two hypotheses we constructed the *Zerbik process calculus*—a fully invented formal system whose symbol vocabulary (zerbik, flarn, quonker, etc.) appears in no pre-training corpus we are aware of and

Table 7: Zerbik process calculus: a synthetic case study we constructed for this paper. The symbol vocabulary is a deliberate non-word set; no memorisation of the derivation is available to the LLM.

#	Abstract	$I(\cdot)$	Real artifact
1	z_1	COMBINE	base + catalyst $\rightarrow$ mixture
2	z_3, z_4	CURE	cured intermediate
3	z_5	DISTILL	distilled solvent
4	z_*	BOND	final compound

whose rules we wrote for this paper. The calculus has seven primitive symbols $\{z_0, \dots, z_7\}$ and a goal symbol z_* ; its six rewrite rules ($\{z_0\} \rightarrow \{z_1, z_2\}$, $\{z_1\} \rightarrow \{z_3\}$, $\{z_2\} \rightarrow \{z_4, z_5\}$, $\{z_3, z_4\} \rightarrow \{z_6\}$, $\{z_5\} \rightarrow \{z_7\}$, $\{z_6, z_7\} \rightarrow \{z_*\}$) define a reachability problem whose solution is the six-step derivation

$$\begin{aligned} \{z_0\} &\rightarrow \{z_1, z_2\} \rightarrow \{z_3, z_2\} \rightarrow \{z_3, z_4, z_5\} \\ &\rightarrow \{z_6, z_5\} \rightarrow \{z_6, z_7\} \rightarrow \{z_*\}. \end{aligned}$$

BFS solves this in 6 rule applications; the LLM-guided V3 solver also recovers a valid derivation (sometimes the same one, sometimes an equivalent ordering). Table 7 gives the interpretation table $I: M_{\text{zbk}} \rightarrow R_{\text{zbk}}$ under which the solved derivation produces a fictional but procedurally coherent industrial-process output; the table is kept offline, as in the other two case studies, and never enters the LLM context.

Scope of the claim and the memorisation question.

The contribution is domain-agnostic pipeline correctness, not severity demonstration. Any domain expressible as a context-free grammar is vulnerable to the same construction; we validate on three structurally distinct domains and decline to validate on high-severity domains by design (§11.2). Two of the three domains (aspirin, TCP) are public knowledge and so in isolation they cannot empirically separate symbolic reasoning from retrieval of memorised sequences. The Zerbik calculus is included precisely to address this confound: its vocabulary is invented, its rules were written for this paper, and the LLM cannot have seen it before—yet the V3 pipeline still recovers a valid derivation, demonstrating that the homomorphic-reasoning construction does not require the LLM to hold prior memory of the target domain. Under either interpretation of what the LLM is “really doing” (symbolic manipulation or pattern-matched retrieval), the inference-layer guardrail sees only abstract grammar symbols and is unable to determine which concrete domain is being instantiated offline. §11.1, item 6 discusses what the synthetic case does and does not rule out.

Visibility trace: what each party sees. Table 8 gives the reviewer-critical observation directly: the LLM’s prompt context and the regex guardrail’s inspected input are *byte-identical abstract grammars*, while the post-interpretation artifact—held only by the attacker, applied offline—is the actual synthesis procedure or protocol run. No content inspection at the LLM boundary can distinguish the two runs, because at the LLM boundary the two runs *are* the same run. This is Observation 6.20 made visible: the guardrail is not poorly configured, it is looking in the only place the dangerous content can never appear.

Defense classification. The aspirin case study is a canonical Class C attack (§10.3): the LLM sees only abstract grammar symbols, never the chemical names or procedures. No inference-layer filter—regex, neural, or any composition thereof—can detect the attack because the prompts are syntactically identical to legitimate grammar exercises (Observation 6.20). The interpretation table $I: M_{\text{chem}} \rightarrow R_{\text{chem}}$ is held offline by the attacker and never appears in any LLM-visible content.

This structure is isomorphic to the `secrets-router` defense (§11.4): in both cases, the security-critical mapping (interpretation table / credential values) is held outside the LLM context, and only abstract references (grammar symbols / opaque handles) are visible to the model. The attack exploits the offline mapping to produce harmful instantiations; the defense exploits the *same* offline-mapping architecture to prevent credential exfiltration. Both follow the abstraction pattern from §4.2: the non-invertibility of Q at the inference layer is a vulnerability when the attacker holds the interpretation table, and a security guarantee when the defender holds it.

The Interpretation Firewall. Our adaptive adversary experiment confirms that inference-layer defenses—even structural analysis combined with re-interpretation probing—fail when the adversary uses opaque symbols and fake recursion (0/3 detection against adaptive Class C attacks, versus 3/3 against naive ones). The general defense architecture is an *interpretation firewall*: rather than inspecting the grammar or derivation (inference layer), inspect the *actions* that follow interpretation (execution layer). Concretely, this requires: (1) action allowlisting (the LLM can trigger only declared tool calls, not arbitrary code); (2) sandboxed interpretation (derivations are instantiated inside an isolation boundary and their effects inspected before release); and (3) provenance tracking (the chain from prompt to grammar to derivation to action is logged and auditable). Tool-use protocols such as MCP already implement (1) via tool declarations and (3) via opaque handles, making them a natural substrate for Class C defense. The remaining gap is (2): MCP tools currently execute with full permissions rather than in a

Table 8: Visibility trace. Columns 1 and 2 are byte-identical abstract grammars—the LLM’s context and the regex guardrail’s inspected input coincide exactly. Column 3 is the post-interpretation artifact, produced offline by applying I to the solved derivation; it is never visible at any LLM boundary. The guardrail is not poorly configured; it is looking in the only place the dangerous content can never appear.

LLM context (prompt)	Guardrail input (inspected bytes)	Post-interpretation artifact
Grammar $S \rightarrow t_1 t_2 t_3 t_4$; solve for reachable t_4	Grammar $S \rightarrow t_1 t_2 t_3 t_4$; solve for reachable t_4	salicylic acid + acetic anhydride $\rightarrow$ crude $\rightarrow$ recrystallize $\rightarrow$ aspirin
Grammar $S \rightarrow t_1 t_2 t_3 t_4 t_5 t_6$; solve for CLOSED	Grammar $S \rightarrow t_1 t_2 t_3 t_4 t_5 t_6$; solve for CLOSED	SYN $\rightarrow$ SYN-ACK $\rightarrow$ ACK $\rightarrow$ ESTABLISHED $\rightarrow$ FIN $\rightarrow$ FIN-ACK $\rightarrow$ CLOSED
Rules $R_1: \{z_0\} \rightarrow \{z_1, z_2\}, \dots$; solve for reachable z_*	Rules $R_1: \{z_0\} \rightarrow \{z_1, z_2\}, \dots$; solve for reachable z_*	combine $\rightarrow$ cure $\rightarrow$ distill $\rightarrow$ bond (Zerbik, synthetic)

sandbox. Closing this gap—sandbox-isolated tool execution with effect auditing—is the single architectural change that would complete the defense stack against all three attack classes.

string. This medium was chosen for its zero-literal property: the AST nodes are constructed programmatically from grammar terminal symbols, so the executed code is never present as a string in the source.

9 Proof-of-Concept Implementation

The PoC implements all five attack vectors across six Python modules: `engine.py` (63 lines: zero-knowledge recurrent executor for V1/V2), `mediums/graph_solver.py` (43 lines: BFS state-space solver), `mediums/tot_solver.py` (158 lines: Tree-of-Thought with LLM evaluation for V3), `mediums/web_scraper.py` (77 lines: AST-based HTTP fetch), `encode.py` (22 lines: config-to-base64 encoder), and a medium registry (`mediums/__init__.py`, 13 lines) that selects the execution medium at runtime. The PoC totals 376 lines of Python—compact enough to fit within a single code review session and below typical automated complexity thresholds. Full source is provided as supplementary material (`supplementary/appendix-poc-source.tex`).

Architecture. `engine.py` decodes a base64-encoded config, selects an AST medium, compiles it via `exec(compile(...))`, injects runtime parameters, and loops until a termination condition. Each medium exports a `build_*_ast()` function returning a Python AST module with zero string literals—all domain content enters via runtime parameters from the opaque config. `tot_solver.py` implements beam search with LLM-guided branch pruning; the LLM (via Ollama) sees only abstract symbols and rates branches as SURE, LIKELY, or IMPOSSIBLE.

AST medium. The derivation output is processed through Python’s AST rather than via `eval` of a raw

Listing 2: PoC CLI usage

```

CLI usage.
# Solve an abstract grammar and
  instantiate
python grammar.py --mode tot --grammar
  path/to/grammar.json \
  --budget 100 --output derivation.json

# Apply interpretation table offline
python engine.py --derivation derivation.
  json \
  --interpretation path/to/I.json --dry-
  run

# Bootstrap grammar from domain
  description
python bootstrap.py --domain "organic
  synthesis" \
  --symbols 12 --output grammar.json

```

The `-dry-run` flag to `engine.py` prints the instantiated operation sequence without executing it, enabling human review before any execution step.

10 Defense Recommendations

Scope of this section (closure under composition). The defense recommendations below rest on a closure-under-composition argument that combines theorems 6.8 and 6.9 (for the regex/star-free side) with the empirical and capacity-bound behavior of TC^0 -class neural filters (theorem 10.1). To make the load-bearing claims explicit:

What we prove. (i) Boolean closure of star-free / aperiodic languages: any finite intersection, union, or comple-

ment of aperiodic substring-matching guardrails remains aperiodic, hence remains in AC^0 , hence remains blind to MOD_p encodings (theorem 6.8, this section’s theorem 6.9). (ii) The closure extends *across defensive-class boundaries*: any composition of star-free string-matchers (regex tier, trie blocklist, finite DFA, Bloom-filter blocklist on its intended literal set) is itself star-free, regardless of which defensive class each constituent comes from (theorem 10.2). (iii) In the regex/neural mixed setting, the four serial/parallel/blend cases of theorem 10.1 characterize when the regex tier’s AC^0 blindness propagates to the composed system and when the neural tier’s TC^0 capacity recovers it.

What we assume. The Boolean-closure model of composition treats a multi-tier defense as a Boolean function of its constituents’ block-decisions (item (3) of theorem 10.1); soft-score serial pipelines are addressed only via the parallel and blend cases. For theorem 10.1(5) we assume the filter score functions $s_r, s_n : \Sigma^* \rightarrow [0, 1]$ are calibrated such that $f(x) = 1 \iff s(x) > \tau_f$ for some threshold τ_f . The neural tier is assumed to be a log-precision transformer, so by Merrill–Sabharwal [25] and Chiang–Cholak–Pillay [5] its decision language lies in TC^0 . The Bloom-filter inheritance argument (theorem 6.9, third bullet) applies to the filter’s *intended* literal set, not to the false-positive superset that the filter actually accepts.

What we leave open. (a) The capacity bound $MOD_p \in TC^0$ used in theorem 10.1(4) is a capacity claim about what a TC^0 circuit *could in principle* compute; it is *not* a guarantee that a specific trained fixed-architecture transformer reliably realises MOD_p on unboundedly long inputs. Hahn [13] shows this gap is real for fixed-architecture self-attention without depth or head scaling; the proposition’s caveat now reflects this. We do not prove that any particular production transformer guardrail closes the bypass class in practice. (b) table 11 is keyed to items (3), (4), and (5) of theorem 10.1; items (1)–(2) (parallel regex and serial regex) follow algebraically from theorem 6.8 and are not reproduced as table rows. (c) Pilot 6 (section 8.2) supplies the empirical leg of theorem 10.2 for two non-LLM defensive classes (WAF and spam filter); the remaining classes named in CACM v14 §2 (literal-string AV signatures, IDS regex tiers, code/secret scanners) remain by algebraic argument only and are listed as future work. (d) Hybrid topologies beyond plain parallel-OR / serial-AND (rate-limited fallback, ensemble voting with a neural majority, sampled-routing, multi-stage agentic pipelines with mixed AC^0/TC^0 arbiters) are discussed informally in CACM v14 §5 and are not formalized here.

10.1 Defense Layer Mapping

Table 9 maps each impossibility result to the defense layer it constrains, the threats that layer can and cannot address,

Table 9: Defense layer mapping. Each row gives the guardrail layer, its mechanism, the threats it can address, and the formal limitation established in this paper.

Layer	Mechanism	Formal limitation
Regex	Substring patterns, blocklists	Provably blind to MOD_p encodings (§6.1–6.2); 100% of corpus aperiodic
Inference	Content filters, RLHF	Illustrative Fano-type lower bound under a uniformity assumption (§6.5); direct requests and obvious encodings caught
Schema	Domain-aware validation	Catches some homomorphic reasoning (high false-positive rate, NP-hard to verify, §6.6)
Execution	Runtime payload inspection, allowlists	Catches ZK pipelines (V2) and instantiated procedures; best defense against V1
OS	Sandboxing, network ACLs	Catches all execution-layer attacks; limits legitimate functionality
Provenance	End-to-end pipeline monitoring	Catches full pipeline attacks; requires offline visibility into attacker-held interpretation table

and the formal bound from this paper.

The key structural insight is that no single layer is sufficient. The Regex layer is algebraically blind (unconditional); the Inference layer faces an illustrative Fano-type lower bound when abstraction fibers are mixed (under the uniformity assumption of §6.5); the Schema layer is NP-hard; and the Execution/OS layers limit functionality. Effective defense requires all layers operating in concert, with particular emphasis on the Execution and Provenance layers, which are the only ones capable of observing the concrete artifacts that carry harmful intent.

10.2 Neural Guardrails and TC^0

Regex guardrails recognize aperiodic languages, placing them in AC^0 via Theorems 5.3–5.5; the algebraic blindness of Theorem 6.4 is the direct consequence. Transformer-based guardrails occupy a strictly higher class: log-precision transformers sit in TC^0 [5, 25], and $AC^0 \subsetneq TC^0$ [9, 14]. Neural guardrails can therefore compute MOD_p predicates that no regex guardrail can; the modular-

counting blind spot of §6.2 does not transfer to them, and a sufficiently trained transformer can in principle detect ϕ_p -encoded payloads.

Table 10: Circuit complexity of guardrail architectures. $AC^0 \subsetneq TC^0$; both are conjectured to be strictly contained in NC^1 .

Architecture	Complexity	Detects MOD_p ?
Regex (substring match)	AC^0	No (Thm 6.4)
Regex (explicit count)	NC^1	Yes, for specific p
1-layer transformer	TC^0	Yes (threshold gates)
Multi-layer transformer	TC^0	Yes, broader class [33]
Full production LLM	Unknown (in TC^0 ?)	Empirically yes

What TC^0 can detect that AC^0 cannot. The threshold gates separating TC^0 from AC^0 compute majority, parity, and iterated addition. A single uniform attention head computes parity on a binary sequence directly: it attends to all positions and accumulates a weighted sum whose threshold distinguishes odd from even counts—the computation AC^0 circuits provably cannot perform [9, 14]. The canonical MOD_2 bypass of Corollary 6.5 (interleaved payload $w = a_1b_1a_2b_2 \cdots a_nb_n$) therefore collapses against a one-layer transformer guardrail while provably evading any aperiodic regex. Strobl et al. [33] give tight formal-language expressivity bounds for hard-attention transformers of each depth, including recognition of non-aperiodic regular languages; Hahn [13] gives parallel circuit-theoretic grounding for the algebraic boundary via soft-attention saturation on periodic languages.

What even TC^0 cannot detect. TC^0 has its own conjectured ceiling [3, 31]: NC^1 -complete problems such as arbitrary Boolean formula evaluation are believed to lie outside TC^0 , though the separation has resisted proof for four decades. A guardrail that must evaluate a complex logical safety policy over its input tokens is conjectured to require NC^1 or higher, and attackers who can force the detection problem into the $NC^1 \setminus TC^0$ gap—conditionally on that separation holding—can evade transformer guardrails by the analogous complexity argument. The algebraic impossibility for regex guardrails (Theorem 6.4) is therefore unconditional, while the analogous statement for neural guardrails is conditional on an open complexity separation. The Fano-type lower bound (Proposition 6.12) and the NP-hardness of instantiation (Proposition 6.14) apply to any guardrail architecture inspecting only LLM-visible content, independently of its circuit class, because both barriers arise from the fiber structure of the abstraction function rather than from the guardrail’s computational complexity.

Empirical TC^0 validation. We trained a minimal transformer classifier—one layer, one attention head,

$d_{\text{model}}=32$, 14,753 parameters—on character-level tokenization of MOD_2 -encoded payloads ($n=30$) versus clean text ($n=30$). On held-out encoded payloads: MOD_2 detection 90%, MOD_3 (unseen encoding) 90%, MOD_5 100%, MOD_7 100%; clean-text FPR 0%. Regex detection: 0% across all encodings. A single attention head with threshold activation detects the interleaving pattern that AC^0 circuits provably cannot compute, empirically confirming the $AC^0 \subsetneq TC^0$ separation in the guardrail domain. Full training metrics, held-out evaluation on MOD_p for $p \in \{2, 3, 5, 7\}$, and clean-text false-positive rates are released in the artifact `results/transformer_tc0_validation.json`.

Practical implications. The hierarchy recommends stacking: regex provides an AC^0 first-pass filter (linear time, parallelizable, provably blind to modular encodings), and a lightweight neural classifier trained specifically for structure-based bypasses [24, 33] closes the MOD_p gap in principle. Recent empirical work confirms the gap exists in production: Zhu et al. [41] report 73.6% attack success via symbolic mathematical encoding across 13 production LLMs, an observable manifestation of the algebraic blindness that Theorem 6.4 explains; cryptographic execution proofs [16] demonstrate a guardrail *ran* but do not address the underlying blindness. Two caveats limit the recommendation. Neural guardrails are three to four orders of magnitude more expensive than DFA traversal, adding non-trivial latency in high-throughput deployments, and transformer guardrails are themselves subject to gradient-based adversarial attacks [42] that operate in weight space rather than in the input language’s algebraic structure—a threat orthogonal to the impossibility results here but independently serious. The illustrative Fano lower bound (Proposition 6.12, under its uniformity assumption) continues to apply at the inference layer regardless of the guardrail’s circuit class; adversarial robustness of neural filters is a separate unsolved problem. Section 10.3 formalizes the correct multi-layer composition as filter composition laws.

10.3 Filter Composition Laws

We prove five composition laws governing the interaction of AC^0 (regex) and TC^0 (neural) filters, answering whether composing guardrail layers can repair the algebraic blindness of §6.2. Let f_r denote a regex filter (star-free language, aperiodic syntactic monoid), f_n a neural filter (TC^0 threshold circuit), and write $f(x) = 1$ if f blocks x .

Proposition 10.1 (Composition ceiling). *Let ϕ_p denote MOD_p encoding.*

1. **Parallel regex preserves blindness:** $(f_{r_1} \vee f_{r_2})(\phi_p(x)) = 0$ for all aperiodic f_{r_1}, f_{r_2} . *Proof:*

Union of star-free languages is star-free (Proposition 6.8); the syntactic monoid of the union remains aperiodic; Theorem 6.4 applies.

2. **Serial regex preserves blindness:** $(f_{r_1} \wedge f_{r_2})(\phi_p(x)) = 0$ for all aperiodic f_{r_1}, f_{r_2} . Proof: Intersection of star-free languages is star-free; same argument.
3. **Serial confirmation preserves blindness (regex veto):** $(f_r \wedge f_n)(\phi_p(x)) = \min(f_r(\phi_p(x)), f_n(\phi_p(x))) = \min(0, f_n(\phi_p(x))) = 0$. The regex filter’s inability to detect the encoded payload vetoes the neural filter’s detection, regardless of the neural filter’s accuracy.
4. **Parallel with neural breaks the ceiling (capacity claim):** $(f_r \vee f_n)(\phi_p(x)) = \max(f_r(\phi_p(x)), f_n(\phi_p(x))) = \max(0, f_n(\phi_p(x))) = f_n(\phi_p(x))$. The composed filter detects if and only if the neural filter detects. Since $f_n \in \text{TC}^0$ and $\text{MOD}_p \in \text{TC}^0$, this composition can in principle escape the AC^0 ceiling. Capacity-vs-realization caveat (Hahn [13]). This is a capacity claim about what some TC^0 circuit could compute, not a guarantee that a specific trained fixed-architecture transformer reliably realises MOD_p on unboundedly long inputs: Hahn 2020 establishes that fixed-architecture self-attention cannot model parity for unboundedly long inputs unless depth or attention heads scale with sequence length. The empirical TC^0 validation of §10.2 confirms that the capacity is achievable in the small-payload regime relevant to deployed guardrails (a 14,753-parameter one-layer-one-head transformer detects MOD_2 on the corpus payload distribution at 90%); the unbounded-length realization remains conditional on architecture scaling.
5. **Critical blend weight.** For a soft-score blend $g_\alpha(x) = 1$ iff $\alpha \cdot s_r(x) + (1 - \alpha) \cdot s_n(x) > \tau$, where $s_r(\phi_p(x)) \approx 0$ (regex blind) and $s_n(\phi_p(x)) = c > 0$ (neural detects), the blend detects $\phi_p(x)$ if $\alpha < \alpha^* := 1 - \tau/c$. Above α^* , the regex weight dominates and the blend inherits AC^0 blindness.

Practical consequence. Items (1)–(2) confirm that stacking multiple regex filters cannot repair blindness. Item (3) shows that a “regex confirm, neural inspect” architecture inherits the regex layer’s blindness, because the dangerous case is precisely the one the regex does not flag. Item (4) gives the correct composition: run both in parallel, block if *either* flags, so the neural classifier covers the algebraic blind spot (subject to the capacity-vs-realization caveat above). Item (5) quantifies the trade-off: soft-score blends must keep $\alpha < \alpha^* = 1 - \tau/c$ or the regex weight dominates and reimports AC^0 blindness.

Remark 10.2 (Cross-class composition: $\text{WAF} \circ \text{IDS} \circ \text{LLM-guardrail}$). Items (1)–(2) and theorems 6.8 and 6.9 extend across defensive-class boundaries. A multi-tier

pipeline composing a WAF rule set, an IDS signature set, and an LLM guardrail — *all* star-free / aperiodic — is itself star-free, by the same Boolean-closure argument (Pin [28] §I.4): the composed system’s syntactic monoid divides the direct product of the three constituents’ syntactic monoids, all of which are aperiodic, so the product is aperiodic and the composition is in AC^0 . The composed system therefore inherits blindness to $L_{\text{decode}}(p)$. The composition trap is the same mathematical object across the defensive-security landscape: serial-AND propagates the most algebraically-constrained tier’s blindness to the system, regardless of which defensive class that tier comes from. Pilot 6 (§8.2) supplies the empirical leg of this cross-class claim for two non-LLM defensive classes (ModSecurity OWASP CRS as the WAF constituent and SpamAssassin 20_phrases.cf as the spam-filter constituent); the remaining classes named in CACM v14 §2 are by algebraic argument only and are listed under “what we leave open” in §10’s scope paragraph. Compositions involving a non-star-free constituent (e.g., a TC^0 neural arbiter in parallel-OR) are addressed by theorem 10.1(4) and lie outside this remark’s scope.

Assumption (model of composition). Items (1)–(4) of theorem 10.1 model multi-tier composition as a *Boolean* function of the constituent filters’ block-decisions ($\wedge$ for serial-AND, $\vee$ for parallel-OR), with $f(x) \in \{0, 1\}$. Item (5) is the soft-score generalisation, with calibrated score functions $s_r, s_n: \Sigma^* \rightarrow [0, 1]$ such that $f(x) = 1 \iff s(x) > \tau_f$; the $s_r(\phi_p(x)) \approx 0$ claim used in item (5) follows from theorem 6.4 applied to a thresholded star-free language. Pipelines outside this Boolean-or-linear-blend model (e.g. neural rerankers conditioning on regex confidence as a feature, or learned routing policies that train on the regex tier’s output distribution) are not covered by theorem 10.1 and are listed under “what we leave open” in §10.

Empirical validation. We validated all five laws on 20 guardrail-style regex patterns, 20 MOD_2 -encoded payloads, and 20 clean-text samples, using a character-bigram centroid classifier as the neural filter (Table 11).

The theoretical critical weight $\alpha^* = 1 - 0.3/0.69 \approx 0.57$ predicts the detect-to-blind transition between $\alpha = 0.5$ and $\alpha = 0.6$, matching the empirical drop from 95% detection to 5%; the serial-confirm row validates Proposition 10.1(3) at zero detection.

Neural evasion analysis. Parallel regex $\vee$ neural closes the AC^0 blind spot but not every attack vector. We tested the neural filter (trained on MOD_2 only) against encoding schemes it had not seen (Table 12).

Attacks cluster into three classes by the minimum defense layer required:

Table 11: Empirical validation of composition laws on 20 encoded payloads. Regex detection rate is 0% on encoded payloads, confirming algebraic blindness. The neural filter achieves 100% detection with 0% FPR. All four composition predictions are confirmed. Per-row detection rates and the per-encoding/per-stack breakdown are archived in `results/comprehensive_empirical_validation.json` (encoding_results, semantic_attacks, and preprocessing keys).

Composition	Detection	FPR	Theorem
Regex only	0%	0%	Thm 6.4
Neural only	100%	0%	TC ⁰
Parallel (OR)	100%	0%	Prop 10.1(4)
Serial confirm (AND)	0%	0%	Prop 10.1(3)
Blend $\alpha=0.4$	100%	0%	Below α^*
Blend $\alpha=0.5$	95%	0%	Near α^*
Blend $\alpha=0.7$	0%	0%	Above α^*

Table 12: Neural filter evasion by attack type. The filter is trained on MOD₂ encoded text only. “Evades Both” indicates attacks that evade the regex $\vee$ neural composition. Per-attack detection rates for the encoding-class rows (MOD_{*p*}, Base64, ROT13) are in `results/comprehensive_empirical_validation.json`; the V1 / V3 adaptive-adversary rows (100% evasion of both regex and neural) are backed by `results/class_c_adaptive_adversary.json` (adaptive_evasion_rate, techniques keys).

Attack	Regex	Neural	Evades Both?
MOD ₂ (trained)	0%	100%	No
MOD ₃ (untrained)	0%	90%	10%
MOD ₅ (diluted)	0%	90%	10%
Base64	0%	90%	10%
ROT13	0%	40%	60%
V1 Decomposition	0%	0%	100%
V3 Abstract grammar	0%	0%	100%
Homoglyphs	4%	60%	40%

1. **Class A (encoding):** MOD_{*p*}, Base64, ROT13. Defended at the inference layer by the neural filter (Proposition 10.1(4)) plus decode-aware preprocessing (attempt common decodings and re-filter the decoded text).
2. **Class B (perturbation):** Homoglyphs, gradient adversarial. Defended by input preprocessing (Unicode NFKC normalization collapses homoglyphs to ASCII equivalents) and classifier ensemble diversity.
3. **Class C (semantic):** V1–V4 homomorphic reasoning. *Cannot* be defended at any inference layer, regardless of computational class. By Observation 6.20, these prompts are syntactically identical to legitimate formal reasoning; any filter that blocks them must also block legitimate use. Defense *requires* execution-layer monitoring (inspect what the LLM *does*, not what it *receives*) or provenance tracking.

This classification sharpens the defense recommendation of Table 9: Classes A and B are solvable at the filter layer with known techniques and low overhead; Class C requires fundamentally different mechanisms that monitor actions rather than content. The practical defense stack is therefore: (0) input preprocessing (normalization and decode attempts), (1) regex first-pass, (2) neural second-pass in parallel (Proposition 10.1(4)), (3) execution-layer action monitoring for semantic attacks.

10.4 The Monoid Extractor as Audit Tool

The 858-line Python monoid extractor released as an artifact of this work takes a guardrail regex and outputs (i) the minimal DFA, (ii) the transition monoid with multiplication table, (iii) all group components identified via standard semigroup algorithms, (iv) the complete blindness spectrum—the set of primes p for which the guardrail is provably MOD_{*p*}-blind (Proposition 6.10), and (v) a concrete MOD₂ bypass witness for each aperiodic pattern. Analysis of the 142 production patterns of §8.1 completes in under 60 seconds with zero false positives, demonstrating practical scalability; the tool integrates naturally into CI/CD as a pre-commit verification step, flagging non-aperiodic patterns before deployment. Because the analysis is theorem-backed rather than heuristic, a clean audit report is a formal guarantee of blindness (to the stated primes), shifting regex-guardrail verification from a confidence-based assertion to a mathematical proof.

Composition safety guarantee. By Proposition 6.8, organizations can verify that their composed guardrail stack (intersection or union of multiple patterns) preserves the blindness property. The product monoid of aperiodic components remains aperiodic, so the audit report for individual patterns generalizes immediately to multi-pattern deployments. If pattern P_1 is aperiodic and pattern P_2 is

aperiodic, then the composed guardrail $P_1 \cap P_2$ (enforce both) or $P_1 \cup P_2$ (enforce either) is provably blind to all modular encodings. This compositional safety is essential for practical systems, which routinely combine blocklists, domain filters, and toxicity detectors.

Empirical validation on 142 patterns. We validated this audit pipeline on the 142 patterns from twelve sources (nine third-party open-source projects plus three author-assembled sets, per CACM v14 Appendix B.1; full inventory in Table 3). All 142 patterns (100%) were aperiodic and therefore provably blind to MOD_p encodings for all primes p . This confirms the conjecture (Section 10.5) that semantic guardrail patterns in practice are aperiodic by structural design, making the monoid extractor a practical tool for organization-wide assurance: “run the extractor, check the output, and deploy with a formal proof of blindness to modular bypass strategies.”

10.5 The Aperiodicity Conjecture

Conjecture 10.3 (Universal aperiodicity). *Every regex pattern used as a semantic content guardrail in practice (keyword blocklists, domain filters, toxicity patterns) has an aperiodic syntactic monoid.*

Evidence. 100% of the 142-pattern corpus is aperiodic. No non-aperiodic patterns were found across any of the twelve sources (nine third-party projects plus three author-assembled sets). All 142 patterns are semantic content filters, consistent with the structural argument that such patterns are aperiodic by construction.

Structural argument. Semantic guardrail patterns are designed to match specific substrings (keywords, domains, toxic phrases). Such patterns are inherently positional and Boolean in character: they ask “does this substring appear?” rather than “does a count of occurrences satisfy a modular condition?” The substring-matching structure is precisely what Theorem 6.1 formalizes, and the proof shows that every pattern of this structural type has an aperiodic monoid. The conjecture amounts to the claim that no semantic guardrail in practice has the structural form of a modular counter, which is supported by both the empirical evidence and the design rationale of practical guardrail systems.

PCRE caveat. This conjecture applies to true regular expressions. PCRE extensions with backreferences recognize context-sensitive languages and fall outside our formal model. Backreferences can encode counting predicates that a pure regex cannot; a PCRE pattern using backreferences to enforce exact character count parity would not have an aperiodic syntactic monoid in the standard

sense. In practice, no guardrail pattern in our corpus used backreferences.

11 Limitations and Ethical Considerations

11.1 Limitations

1. Test case severity. The empirical validation uses the aspirin synthesis case study, chosen for its minimal harm risk. The formal results (Theorems 6.1 and 6.4, Proposition 6.17, and Observation 6.20) are domain-independent; their applicability to high-severity domains is implied by the theory but not empirically demonstrated here by design.

2. ToT scope. The Tree-of-Thought benchmark evaluates 50 randomized grammars (seeds 0–49). This is a moderate sample; the yield and solve-rate estimates carry bootstrap 95% confidence intervals reported in Table 4. Systematic variation with grammar depth, branching factor, and symbol set size remains to be characterized.

3. Uniformity assumption. The Fano bound (Proposition 6.12) assumes a uniform distribution over the concrete artifact space R . In practice, the distribution over prompts is highly non-uniform; the bound is tight under uniformity and may be loose or conservative under real prompt distributions. Under highly skewed distributions where adversarial prompts are rare, the conditional entropy $H(G(R)|U(R))$ may be near zero, making the Fano floor uninformative. Conversely, if even a small fraction of mass lands in mixed fibers (concrete artifacts mapping to both safe and unsafe abstract types), the floor remains nonzero. A distribution-aware version of the bound would require modeling the attacker’s prompt distribution; we leave this as future work, noting that the *algebraic* barriers (Theorems 6.1–6.4) are unconditional and do not depend on any distributional assumption.

4. Regex corpus size. The corpus of 142 patterns from twelve sources (nine third-party projects plus three author-assembled sets, per CACM v14 Appendix B.1) is representative but not exhaustive. A larger corpus would strengthen the aperiodicity conjecture (Section 10.5). We make the monoid extractor available to enable independent extension of the corpus.

5. Abstract scope. The algebraic impossibility results (Theorems 6.1 and 6.4) apply specifically to *substring-matching regex guardrails*. They do not apply to all regular expression uses, to neural guardrails (which operate in TC^0), or to guardrails that perform semantic analysis beyond pattern matching.

6. Case-study memorization confound. Two of the three end-to-end case studies (aspirin synthesis, TCP state machine) are public-knowledge domains extensively represented in LLM pre-training data, raising a legitimate confound: those runs in isolation cannot empirically distinguish homomorphic reasoning over abstract grammars from retrieval of memorised sequences keyed off recognisable symbolic patterns. The third case study (the Zerbik process calculus of §8.4, Table 7) was constructed specifically to control for this confound: its symbol vocabulary is invented, its rewrite rules were written for this paper, and the LLM cannot have seen it in pre-training; the V3 pipeline nonetheless recovers a valid derivation, demonstrating that the homomorphic-reasoning construction does not require prior memorisation of the target domain. Beyond that controlled case, the formal contribution does not depend on the LLM actually performing symbolic reasoning in any interpretive sense: the functoriality of the quotient interpretation map (Proposition 6.17) and the structural indistinguishability observation (Observation 6.20) require only that the LLM-visible surface of the attack be syntactically indistinguishable from a legitimate formal-reasoning workload, which Table 8 demonstrates directly across all three case studies. The Zerbik case study is a single invented domain; further scale-up (many synthetic grammars parameterised by rule count, branching factor, and symbol-set size) would strengthen the empirical evidence beyond this point and is left as future work.

11.2 Ethical Considerations

The impossibility results in this paper demonstrate structural limitations of LLM guardrail systems. We have taken the following precautions.

Validation without high-severity cases. Empirical validation uses only the aspirin synthesis case study and abstract grammar benchmarks. We do not demonstrate the pipeline on domains with high harm potential.

Responsible disclosure. The existence of modular counting bypasses against regex guardrails is an implication of classical algebraic results (Theorem 6.4); any party aware of the Schützenberger–McNaughton–Papert theorem and its circuit complexity consequences could derive this result independently. We therefore consider the algebraic results non-sensitive from a disclosure perspective. The V3/V4 pipeline design is more operationally specific; we describe it at the level of algorithmic structure rather than providing a production-ready exploit.

Defense-first framing. Every impossibility result in this paper is paired with a specific defense recommendation in Section 10. The paper is structured to enable defenders

to audit their systems using the monoid extractor and to migrate critical enforcement to execution-layer defenses.

The `exec(compile())` irony. We note the irony that the PoC’s `engine.py` uses `exec(compile(...))`—a well-known security antipattern—as its core execution mechanism. This is by design: the zero-literal AST compilation is precisely the technique that enables the attack. Highlighting this irony is intended to motivate defenders to treat dynamic code construction patterns as a first-class detection target at the execution layer.

11.3 End-to-End Validation Against Production Patterns

We tested the full defense stack against 23 patterns extracted from LLM Guard’s `BanSubstrings` scanner [29], a production guardrail library maintained by ProtectAI. Table 13 compares four defense configurations across 8 encoding types on 20 payloads spanning prompt injection, code injection, SQLi, and XSS.

Table 13: End-to-end detection rates (%) against real LLM Guard patterns. “Regex Only” is the current production configuration. “Full Stack” adds preprocessing (strip, decode, normalize) and a bigram neural filter in parallel. Source: `results/e2e_llmguard_validation.json` (`n_patterns=23`, `n_payloads=20`, `patterns_source = LLM-Guard BanSubstrings`).

Encoding	Regex	+Preproc	+Neural	Full Stack
Raw payload	95	95	95	95
MOD ₂	0	0	100	100
MOD ₃	0	0	100	100
Base64	0	95	100	100
ROT13	0	95	90	100
ZWSP	0	95	0	95
Homoglyph	15	95	60	95
Leetspeak	0	95	80	100

The “Regex Only” column confirms the algebraic blindness theorem in practice: LLM Guard’s production patterns detect 0% of encoded payloads across all seven encoding types. The “Full Stack” column shows that our recommended defense—preprocessing plus parallel neural filter—recovers 95–100% detection on all encodings. The defense gap is entirely attributable to the AC^0 ceiling (Theorem 6.4), and the recovery is entirely attributable to the ceiling-breaking composition (Proposition 10.1(4)) and invertibility-based preprocessing.

11.4 Case Study: secrets-router as Execution-Layer Defense

The three-class attack taxonomy (§10.3) establishes that Class C (semantic/homomorphic reasoning) attacks *cannot* be defended at any inference layer. We present `secrets-router`¹, an open-source MCP server that implements the execution-layer defense our theory recommends.

Architecture. When an AI agent needs a credential, it calls `secure_fetch("rbw", "primary card", "number")` and receives an opaque handle `handle:a3f8c2d1`. The credential value never enters the LLM context window. A separate server process resolves handles via Chrome DevTools Protocol (CDP), filling browser fields directly at the execution layer. The agent sees only masked feedback: `"filled [MASKED ****1017]"`.

Connection to the theory. The opaque handle mechanism implements the abstraction pattern from our formalization (§4.2): it maps concrete credentials to abstract handles, and the non-invertibility at the inference layer provides the security guarantee. The CDP fill operates at the execution layer (Table 9, row 4), precisely the layer our theory identifies as necessary for Class C defense. Of the three components the Interpretation Firewall requires (§8.4: action allowlisting, sandboxed interpretation, provenance tracking), MCP natively provides two—tool declarations implement allowlisting, and opaque handles implement provenance—leaving sandboxed interpretation as the only remaining gap.

Defense walkthrough against V3/V4. The symmetry with our own attack is exact. An attacker running V3 or V4 against a system protected by `secrets-router` cannot exfiltrate credentials: the LLM context contains only opaque handles, the interpretation table $I: M_{\text{cred}} \rightarrow R_{\text{cred}}$ is held by the defender and never appears on an LLM-visible surface, and the resolution of handles to concrete values happens at the execution layer after every inference-layer inspection has already passed. The homomorphic-reasoning attack of §7.3 exploits the defender’s inability to see the interpretation table while the attacker holds it; `secrets-router` inverts this asymmetry by putting the defender on the held-offline side of the abstraction boundary. The attack and the defense are instances of the same architectural pattern, distinguished only by who holds the interpretation function I (cf. Table 8, row structure):

¹<https://jrlopez.dev/p/secrets-router.html>, MIT license, ~800 lines. *Disclosure: this implementation was developed by the author as part of this project. Readers should verify its properties against their own threat model.*

the non-invertibility is a vulnerability when the attacker holds I and a security guarantee when the defender holds it.

12 Conclusion

We have established four independent, formally grounded impossibility results for LLM guardrail systems.

Algebraic impossibility (regex layer). All substring-matching regex guardrails have aperiodic syntactic monoids (Theorem 6.1). By the Schützenberger–AC⁰–Furst–Saxe–Sipser chain, aperiodic syntactic monoids characterize exactly the languages in AC⁰, which cannot compute modular counting predicates. Therefore, any regex guardrail is provably blind to modular counting encodings (Theorem 6.4), and this blindness cannot be patched within the aperiodic regex paradigm (Corollary 6.5). The Krohn–Rhodes decomposition gives a constructive audit tool: the blindness spectrum of any guardrail is computable from its syntactic monoid’s group components (Proposition 6.10). Our corpus study confirms 100% aperiodicity in practice.

Information-theoretic lower bound (inference layer). Abstraction functions that map concrete artifacts to abstract types inevitably create mixed fibers. Under a uniform prior over concrete artifacts, Fano’s inequality yields an illustrative misclassification floor for any lifted guardrail: with 30% of mass in mixed fibers, the floor is approximately 5% (Proposition 6.12). This is an illustrative bound, not an unconditional impossibility; the algebraic blindness result above is the core contribution, and the Fano bound quantifies an intuition about mixed fibers under simplifying assumptions.

Computational impossibility (schema layer). Verifying whether an abstract program has a harmful instantiation is NP-complete (Proposition 6.14). Exhaustive checking is therefore infeasible, and heuristic checking inherits the false-negative rate implied by the combinatorial search space.

Structural transfer and indistinguishability. Abstract derivations transfer faithfully to real-domain operation sequences via free-category functoriality (Proposition 6.17, unconditional): LLM reasoning over abstract grammars produces correct real-world procedures after offline instantiation, regardless of whether the LLM’s internal process is interpretive reasoning or memorized retrieval. Under Assumption 6.19 on symbol naming, the set of adversarial prompts issued by the homomorphic-reasoning pipeline is contained (as strings) in the set of legitimate

formal-reasoning prompts over the same grammar (Observation 6.20); a guardrail inspecting only prompt content therefore has zero distinguishing advantage between the two classes. We present the indistinguishability result as a structural closure of the attack construction under opacity, not as an independent impossibility theorem—the work is done by the opacity assumption and by the *existence* of attacks satisfying it, demonstrated empirically in §7.3–7.4.

Empirical grounding. Five attack vectors demonstrate both barrier classes. The Tree-of-Thought benchmark ($N = 50$ grammars, seeds 0–49, $p < 0.001$) shows that BFS exhaustive search dominates LLM-guided ToT on small synthetic grammars (mean yield 0.466 vs. 0.122), confirming that the security claim of V3 depends on the existence of a solver—not on LLM superiority over BFS. The aspirin synthesis case study validates end-to-end pipeline correctness in a benign domain.

Practical consequence. Inference-layer guardrails are necessary but insufficient. The formal results collectively imply that safety enforcement cannot be achieved at the content-inspection layer alone: the barriers are structural, not implementation defects. Effective defense must migrate critical enforcement to the execution layer, where concrete artifacts—decoded strings, instantiated parameters, evaluated ASTs—become observable. The Provenance layer (end-to-end pipeline monitoring) provides the only defense against the full attack chain, at the cost of requiring offline visibility into attacker-held interpretation tables. This is a fundamental asymmetry: the attacker’s interpretation table is not a secret key whose existence can be assumed away, but an intrinsic feature of the homomorphic reasoning construction. Defense-in-depth across all six layers of Table 9 is the only configuration that addresses all four barrier classes simultaneously.

References

- [1] D. Angluin. Learning regular sets from queries and counterexamples. *Information and Computation*, 75(2):87–106, 1987.
- [2] Sarah Ball, Shafi Goldwasser, Omer Reingold, and Guy Rothblum. On the impossibility of separating intelligence from judgment: The computational intractability of filtering for AI alignment. *arXiv preprint arXiv:2507.07341*, 2025.
- [3] D. A. M. Barrington, K. Compton, H. Straubing, and D. Thérien. Regular languages in NC^1 . *Journal of Computer and System Sciences*, 44(3):478–499, 1992.
- [4] N. Boucher, I. Shumailov, R. Anderson, and N. Papernot. Bad characters: Imperceptible NLP attacks. In *IEEE Symposium on Security and Privacy*, 2022.
- [5] D. Chiang, P. Cholak, and A. Pillay. Tighter bounds on the expressivity of transformer encoders. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, 2023. arXiv:2301.10743.
- [6] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- [7] S. Eilenberg. *Automata, Languages, and Machines, Volume B*. Academic Press, 1976.
- [8] M. K. B. Esmradi, R. Nair, et al. h4rm3l: A dynamic benchmark of composable jailbreak attacks for LLM safety assessment. *arXiv preprint arXiv:2408.04811*, 2024.
- [9] M. Furst, J. B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. In *Proceedings of the 22nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 260–270, 1981.
- [10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [11] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In *Proceedings of the First Workshop on LLM Security (LLMSEC), co-located with ACL 2025*, 2025. arXiv:2504.11168.
- [12] T. Hagendorff, E. Derner, and N. Oliver. Large reasoning models are autonomous jailbreak agents. *Nature Communications*, 2026.
- [13] M. Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.
- [14] J. Håstad. *Computational Limitations of Small-Depth Circuits*. PhD thesis, MIT, 1987.
- [15] N. J. Hopper, J. Langford, and L. von Ahn. Provably secure steganography. In *Advances in Cryptology — CRYPTO 2002*. Springer, 2002.

- [16] Xisen Jin, Michael Duan, Qin Lin, Aaron Chan, Zhenglun Chen, Junyi Du, and Xiang Ren. Proof-of-guardrail in AI agents and what (not) to trust from it. *arXiv preprint arXiv:2603.05786*, 2026. Proves the guardrail ran; our paper proves the guardrail is blind even when it provably runs. Complementary results.
- [17] K. Krohn and J. Rhodes. Algebraic theory of machines. I. Prime decomposition theorem for finite semigroups and machines. *Transactions of the American Mathematical Society*, 116:450–464, 1965.
- [18] A. Kumar, C. Agarwal, S. Srinivas, G. Li, S. Feizi, and H. Lakkaraju. Certifying LLM safety against adversarial prompting. *arXiv preprint arXiv:2309.02705*, 2023. Proposes certified defenses based on randomized smoothing applied to LLM input representations, providing provable safety guarantees.
- [19] Divyanshu Kumar et al. No free lunch with guardrails. *arXiv preprint arXiv:2504.00441*, 2025. Demonstrates that guardrail overhead imposes non-trivial computational cost and that no single guardrail strategy dominates across threat models.
- [20] Bangxin Li, Hengrui Xing, Cong Tian, Chao Huang, Jin Qian, Huangqing Xiao, and Linfeng Feng. StructuralSleight: Automated jailbreak attacks on large language models utilizing uncommon text-organization structures. *arXiv preprint arXiv:2406.08754*, 2024.
- [21] Y. Li et al. SoK: Evaluating jailbreak guardrails for large language models. *arXiv preprint arXiv:2506.10597*, 2025. Systematization-of-knowledge paper evaluating the effectiveness and limitations of current jailbreak guardrail approaches across categories.
- [22] S. Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer-Verlag, 1971.
- [23] R. McNaughton and S. Papert. *Counter-Free Automata*. MIT Press, 1971.
- [24] W. Merrill. What formal languages can transformers express? A survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 2024.
- [25] W. Merrill and A. Sabharwal. The parallelism trade-off: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.
- [26] J. Myhill. Finite automata and the representation of events. *WADD Technical Report*, 57–624, 1957.
- [27] A. Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.
- [28] J.-É. Pin. *Varieties of Formal Languages*. Plenum Press, 1986.
- [29] ProtectAI. LLM Guard: The security toolkit for LLM interactions. <https://github.com/protectai/llm-guard>, 2024. Open-source input/output scanner library for LLM applications; provides a regex-based BanSubstrings scanner and neural content filters. Accessed 2026.
- [30] M.-P. Schützenberger. On finite monoids having only trivial subgroups. *Information and Control*, 8(2):190–194, 1965.
- [31] M. Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012.
- [32] H. Straubing. *Finite Automata, Formal Logic, and Circuit Complexity*. Birkhäuser, 1994.
- [33] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. Formal language recognition by hard attention transformers: Precise bounds and some separations. *Transactions of the Association for Computational Linguistics*, 12:1–19, 2024.
- [34] Xiaobing Sun, Perry Lam, Shaohua Li, Zizhou Wang, Rick Siow Mong Goh, Yong Liu, and Liangli Zhen. Structured semantic cloaking for jailbreak attacks on large language models. *arXiv preprint arXiv:2603.16192*, 2026. Concurrent work.
- [35] Johan Wahréus, Ahmed Hussain, and Panos Papadimitratos. Jailbreaking large language models through content concretization. *arXiv preprint arXiv:2509.12937*, 2025.
- [36] A. Wei, N. Haghtalab, and J. Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [37] Y. Wei et al. DictAttack: Exploiting key-value dictionaries for jailbreaking large language models. *arXiv preprint arXiv:2503.24191*, 2025.
- [38] G. Weiss, Y. Goldberg, and E. Yahav. Extracting automata from recurrent neural networks using queries and counterexamples. *Machine Learning*, 111:2855–2882, 2022.
- [39] Z. Zeng et al. R2-Guard: Robust reasoning-based guardrail for LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. Introduces

a reasoning-based guardrail using chain-of-thought evaluation to improve detection of harmful content over regex and embedding approaches.

- [40] Y. Zhou et al. STAC: When innocent tools form dangerous chains. *arXiv preprint arXiv:2509.25624*, 2025.
- [41] M. Zhu et al. MathPrompt: Mathematical encoding attacks against LLM safety. *arXiv preprint arXiv:2409.11445*, 2024. 73.6% attack success rate via symbolic math encoding across 13 LLMs. Our algebraic blindness theorem explains why this works.
- [42] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Agentic Research Methodology (Supplementary)

This paper was developed using an agentic AI research pipeline. Full pipeline details, skill definitions, and reproduction scripts are available at the project repository. We document the methodology both as a reproducibility aid and because the Five-stage pipeline: (1) human-directed threat model and proof targets; (2) delegated code generation via local LLM agents (qwen2.5-coder:7b); (3) adversarial mathematical audit by a separate AI reviewer instance; (4) three-round simulated peer review (V1 Reject → V2 Weak Accept → V3 Accept with minor revisions); (5) consolidation into current paper.

The peer reviewer caught a BFS-vs-ToT attribution error during V1 review, forcing the honest reframe in Section 8.3—the same formal reasoning capability that enables the attacks in this paper also enables quality gates in AI-assisted research.

Reproducibility. All code, prompts, and model outputs are available at the project repository. Models: Claude Opus 4.6 (architecture), Claude Haiku 4.5 (code generation), qwen2.5-coder:7b (benchmarks), all via Ollama or Anthropic API as of March 2026.